SRINIVAS UNIVERSITY

INSTITUTE OF COMPUTER SCIENCE AND INFORMATION SCIENCES

THIRD SEMESTER QUESTION BANK

COURSE NAME: DATA STRUCTURE USING C++

COURSE CODE: 25BCA31

CREDITS: 4

UNIT 1

1. What is a step-by-step representation of a solution to a given problem called?

A. Algorithm
B. Flowchart
C. Pseudocode
D. None of the above
Answer: A. Algorithm

2. Which of the following is a linear data structure?

A. Graph

B. Tree

C. Stack

D. Heap

ANSWER: C. Stack

3 Which data structure would you use to solve the Towers of Hanoi problem using recursion?

A. Tree

B. Heap

C. Priority Queue

D. Stack

Answer: D. Stack

4 What is the factorial of a positive integer n?

A. n!=n(n-2)!

B. n!=n(n+2)!

C. n!=(n-2)!

D. n! = n(n-1)!

Answer D. n! = n(n-1)!

5

Which of the following problems cannot be solved using recursion?

A. Factorial of a number

B. Nth fibonacci number

C. Length of a string

D. Problems without base case

Answer D. Problems without base case

6

Which of the following is a non-linear data structure?

A. Stack

B. Queue

C. Tree

D. Array

Answer: C. Tree

7

Which statement is used to terminate an algorithm?

A. Stop

B. Exit

C. Return

D. Break

ANSWER: B. Exit

8

Which type of control structure is used to execute a block repeatedly?

A. Sequential

B. Selection

C. Iteration

D. Decision

ANSWER C. Iteration

9 In the Tower of Hanoi problem, how many pegs are used?

A. 2

B. 3

C. 4

D. 5

ANSWER: B. 3

10 Which recursive technique generates the sequence 0, 1, 1, 2, 3, 5, ...?

A. Bubble Sort

B. Tower of Hanoi

C. Fibonacci

D. Binary Search

ANSWER: C. Fibonacci

Objective Type of Questions (MCQ)

11 In the Tower of Hanoi problem, three disks are moved from one peg to another. What is the minimum number of moves?

A) 5
B) 6
C) 7
D) 8
Answer: C) 7

12 Consider the following operations on a data structure:

Insert(10)

Insert(20)

Insert(30)

Delete()

If the structure follows FIFO, which value will be deleted?

A) 10
B) 20

C) 30
D) None

Answer: A) 10

13 If you create an array to store 3 elements and indexing starts from 0, which indices would you use?
A. 1, 2, 3
B. 0, 1, 2
C. 0, 1, 3
D. 1, 2, 4

Answer: B. 0, 1, 2

14 If you need to store multiple elements of the same data type in a program, which data structure would you use?
A. Tree
B. Graph
C. Array
D. Stack

Answer: C. Array

15 If you need to remove an existing element from a data structure, which operation would you perform?

A. Insertion

B. Traversal

C. Deletion

D. Sorting

Answer: C. Deletion

16 If you need to represent relationships where data items are arranged in a parent-child hierarchy, which data structure would you use?

A. Array

B. Queue

C. Tree

D. Stack

Answer: C. Tree

17 A recursive function calculates factorial(4) using 4 × factorial(3). What value is returned here?
A) 12
B) 16
C) 24
D) 36
Answer: C) 24

18 If a program needs to execute different statements depending on whether a condition is true or false, which control structure would you use?

A. Sequential

B. Iteration

C. Selection

D. Traversal

Answer: C. Selection

19 While designing a recursive function, what would you define to stop the recursive calls?

A. Recursive call

B. Base case

C. Iteration

D. Selection

Answer: B. Base case

20 For the Fibonacci sequence 0, 1, 1, 2, 3, 5, ..., what is the next value?
A) 6
B) 7
C) 8
D) 9
Answer: C) 8

Short Answer Questions (3 Marks)

21. What is a Data Structure? Give example.

Answer:

A data structure is a method of storing and organizing data in a computer so that it can be used efficiently. It represents the logical or mathematical model of a particular organization of data.

Examples:

- Array
- Linked List
- Stack
- Queue
- Tree
- Graph

Data structures help in efficient storage, retrieval and manipulation of data.

22. Explain the classification of Data Structures.

Answer:

Data structures are generally classified into **Primitive** and **Non-Primitive** data structures.

1. **Primitive Data Structures:**
   These can be manipulated directly by machine instructions.
   Examples: Integer, Real, Character and Boolean.
2. **Non-Primitive Data Structures:**
   These cannot be manipulated directly by machine instructions.
   Examples: Linked List, Stack, Queue, Tree and Graph.

Non-primitive data structures are further classified into:

- **Linear:** Array, Linked List, Stack and Queue.
- **Non-Linear:** Tree and Graph.

23. What are the basic Data Structure operations?

Answer:

The basic operations performed on data structures are:

1. **Traversing:** Accessing each record exactly once so that the required items can be processed.
2. **Searching:** Finding the location of a record with a given key value.
3. **Insertion:** Adding a new record to a data structure.
4. **Deletion:** Removing a record from a data structure.
5. **Sorting:** Arranging records in a logical order.
6. **Merging:** Combining records from two different sorted files into a single sorted file.

24. Define the following terms:

i. Data
ii. Data Item
iii. Information

Answer:

i. Data:
Data are simply values or sets of values.

ii. Data Item:
A data item refers to a single unit of values. For example, a phone number can be treated as a single data item.

iii. Information:
Information is data with given attributes or meaningful/processed data.

25. What is an Algorithm? State any four characteristics of a good algorithm.

Answer:

An algorithm is a set of well-defined instructions used to solve a particular problem. It takes input and produces the desired output.

Characteristics of a good algorithm:

1. It should have clearly defined input.
2. It should produce a definite output.
3. Every step should be clear and unambiguous.
4. It should terminate after a finite number of steps.

26. What is sub algorithm? Explain.

Answer:

A **sub algorithm** is a complete and independently defined algorithmic module used by a main algorithm or another sub algorithm.

A sub algorithm receives values called **arguments** from the calling algorithm, performs the required computations and sends the result back to the calling algorithm.

There are two types of sub algorithms:

1. **Function sub algorithm** – returns only a single value.

2. **Procedure sub algorithm** – may send back more than one value.

For example, a sub algorithm called **MEAN** can calculate the average of three numbers.

27. Explain the basic control structures used in algorithms.

Answer:

The three basic control structures used in algorithms are:

1. **Sequential Logic:**
   Instructions are executed one after another in a sequential order.
2. **Selection Logic:**
   A condition is tested and one of the alternative modules is selected for execution. It includes if structures.
3. **Iteration Logic:**
   A set of instructions is repeated until the specified condition becomes false. Examples are **Repeat-For** and **Repeat-While** loops.

28. What is Recursion? Mention any two recursive techniques.

Answer:

Recursion is a technique of defining a set or process in terms of itself. A recursive procedure can call itself directly or indirectly.

A recursive procedure must have two important properties:

1. There must be a **base criterion** for which the procedure does not call itself.
2. Each recursive call must move closer to the base criterion.

Two recursive techniques are:

- Factorial
- Fibonacci sequence

29. Explain the Tower of Hanoi problem.

Answer:

The Tower of Hanoi is a mathematical puzzle consisting of **three pegs** called Source, Auxiliary and Destination, with a number of disks arranged on the source peg.

The objective is to transfer all the disks from the source peg to the destination peg while maintaining the same order.

The rules are:

1. Only one disk can be transferred at a time.
2. Only the uppermost disk of a peg can be moved.
3. A larger disk must never be placed on a smaller disk.

The problem can be solved using recursion.

30. What is a recursive function? State and explain its two essential properties.

Answer:

A recursive function or procedure is one which calls itself directly or indirectly.

A recursive procedure must have the following two essential properties:

1. **Base Criterion:**
   There must be a condition for which the procedure does not call itself. This stops the recursion.
2. **Progress Towards Base Criterion:**
   Every time the procedure calls itself, the new problem must be closer to the base criterion.

For example, in factorial:

- If n = 0, then 0! = 1 — base case.
- If n > 0, then n! = n × (n-1)! — recursive case.

Thus, a recursive procedure with these two properties is called well-defined.

Long Answer Questions (5 Marks)

31. Define Data Structure and discuss its importance.

Answer:

A **data structure** is a method of storing data in a computer so that it can be used efficiently. The logical or mathematical model of a particular organization of data is called a data structure.

Data structures mainly deal with:

- How data is organized in memory.
- How efficiently data can be stored in memory.
- How efficiently data can be retrieved and manipulated.

- How different data items are logically related.

Importance of Data Structures:

1. Data structures allow information to be stored and organized.
2. They provide a means for managing large datasets such as databases.
3. They are necessary for designing efficient algorithms.
4. They allow safe storage of information on a computer.
5. They allow data to be used and processed by software systems.
6. They make data processing easier.
7. They are useful for applications such as fast data lookup, scheduling, dictionary indexing, dynamic memory allocation and directory traversal.

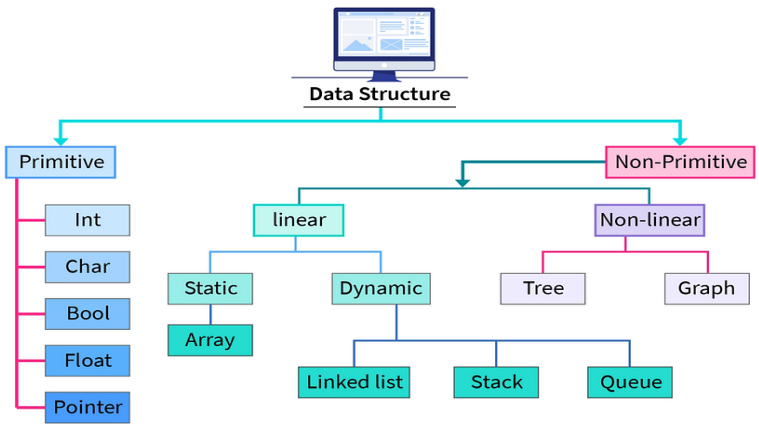
Thus, data structures provide an efficient way of organizing, storing and processing data.

32. List and explain types of Data Structures with a neat diagram and examples.

Answer:

Data structures are generally classified into **Primitive** and **Non-Primitive** data structures.

Classification:



1. Primitive Data Structures

Primitive data structures can be manipulated directly by machine instructions.

Examples:

- Integer
- Real
- Character

- Boolean

These are also called simple data types because they cannot be divided further.

2. Non-Primitive Data Structures

Non-primitive data structures cannot be manipulated directly by machine instructions. They are used to organize more complex data.

Examples include:

- Linked List
- Stack
- Queue
- Tree
- Graph

Non-primitive data structures are further classified as:

a) Linear Data Structures:
The elements form a sequence or linear list.

Examples:

- Array
- Linked List
- Stack
- Queue

b) Non-Linear Data Structures:
The data is not arranged in a sequence.

Examples:

- Tree
- Graph

33. Explain the various Data Structure operations with suitable examples.

Answer:

The basic operations performed on data structures are as follows:

1. Traversing

Traversing means accessing each record exactly once so that the required items can be processed.

Example: Displaying every element of an array.

2. Searching

Searching means finding the location of a record with a given key value or finding records satisfying a particular condition.

Example: Finding a student's roll number in a list of students.

3. Insertion

Insertion means adding a new record or data item to a data structure.

Example: Adding a new student record to a list.

4. Deletion

Deletion means removing an existing record from a data structure.

Example: Removing a student record from a list.

5. Sorting

Sorting means arranging records in a particular logical order.

Example: Arranging student marks in ascending order.

6. Merging

Merging means combining records from two different sorted files into a single sorted file.

Example: Combining two sorted lists into one sorted list.

Thus, these operations are used for efficient management, processing and organization of data.

34. Define array, queue, stack, linear and non linear data structures.

Answer:

1. Array

An **array** is a fixed-size sequential collection of elements of the same type. A specific element of an array is accessed using an index. Array elements are stored in contiguous memory locations.

Example:

A = [10, 20, 30, 40]

2. Queue

A **queue** is a linear list in which deletion takes place at one end called the **front**, while insertion takes place at the other end called the **rear**.

A queue follows the **FIFO (First In First Out)** principle.

Example:

Front → 10 20 30 ← Rear

3. Stack

A **stack** is a linear list in which insertion and deletion take place only at one end called the **top**.

A stack follows the **LIFO (Last In First Out)** principle.

Example:

TOP

↓

30

20

10

4. Linear Data Structure

A data structure is called linear when its elements form a sequence or linear list.

Examples: Array, Linked List, Stack and Queue.

5. Non-Linear Data Structure

A data structure is called non-linear when the data is not arranged in a sequence.

Examples: Tree and Graph.

35. Define data structure and write its advantages.

Answer:

A **data structure** is a method of storing data in a computer so that it can be used efficiently. It organizes data in a logical manner and provides methods for storing, retrieving and manipulating the data.

Advantages of Data Structures:

1. Data structures allow information to be stored on hard disks.
2. They provide a means for managing large datasets such as databases and internet indexing services.
3. They are necessary for designing efficient algorithms.

4. They allow safe storage of information on a computer for later use.
5. They allow data to be used and processed by software systems.
6. They make processing of data easier.
7. Data can be accessed using connected machines through the internet.

Practical uses of data structures include fast data lookup, processing scheduling, dictionary indexing, IP addressing, parsers/compilers, dynamic memory allocation, directory traversal and organizing file hierarchy.

Thus, data structures are important for efficient storage, processing and management of data.

36. Explain Algorithmic Notations with suitable examples.

Answer:

Algorithmic notations are the standard conventions used for presenting the steps of an algorithm in a clear and systematic manner.

The important algorithmic notations given in the study material are:

1. Steps, Control and Exit

The steps of an algorithm are executed one after another, beginning with Step 1. The **Exit** statement indicates that the algorithm is completed. Control can also be transferred to another step using a **Go To** statement.

2. Comments

Comments are written in brackets and indicate the main purpose of a particular step.

Example:

Step 1: [Initialize]

3. Variable Names

Variables are used to store values. Examples are MAX, LOC and K.

4. Assignment Statement

The assignment statement is used to assign a value to a variable.

Example:

MAX := A[1]

This assigns the value of A[1] to MAX.

5. Input and Output

The **Read** statement is used to input data.

Read(A)

The **Write** or **Print** statement is used to display output.

Write(A)

6. Procedures

A procedure is an independent algorithmic module that solves a particular problem.

Example:

Procedure MEAN(A, B, C)

1. Set AVE := (A+B+C)/3

2. Return(AVE)

Thus, algorithmic notations provide a systematic way of writing and representing algorithms.

37. Describe the different control structures with example.

Answer:

Control structures determine the flow of execution in an algorithm. There are three main types of control structures:

1. Sequential Logic

In sequential logic, instructions are executed one after another in a sequential order.

Example:

Step 1: Read A, B

Step 2: Set SUM := A + B

Step 3: Write SUM

Here, the statements are executed in the given order.

2. Selection Logic

Selection logic uses conditions to select one among different alternative modules.

It includes:

- Single alternative
- Double alternative
- Multiple alternatives

Example:

If condition, then

Module A

Else

Module B

End of if

If the condition is true, Module A is executed; otherwise Module B is executed.

3. Iterative Logic

Iterative logic is used to repeat a set of instructions.

There are two types:

a) Repeat-For

Repeat for K = R to S by T

Module

End of loop

The loop continues by changing the value of K according to the specified increment.

b) Repeat-While

Repeat while condition

Module

End of loop

The loop continues as long as the condition is true and stops when the condition becomes false.

Thus, sequential, selection and iteration structures control the flow of an algorithm.

38. What is the recursive technique for generating the Fibonacci series? Explain.

Answer:

The **Fibonacci sequence** is:

0, 1, 1, 2, 3, 5, 8, 13, ...

The first two values are:

$F_0 = 0$

$F_1 = 1$

Each succeeding term is the sum of the two preceding terms.

The recursive definition is:

If n = 0 or n = 1,

$F_n = n$

If n > 1,

$F_n = F_{n-2} + F_{n-1}$

The base values are **0 and 1**. Each recursive calculation uses smaller values of n, which move closer to the base values.

Recursive Procedure

Procedure FIBONACCI(Fib, n)

1. If n = 0 or n = 1 then
2. Set Fib := n and return
3. Call Fibonacci(Fib a, n-2)
4. Call Fibonacci(Fib b, n-1)
5. Set Fib := Fib a + Fib b
6. Return

Example

To find F_5:

$F_5 = F_3 + F_4$

$F_3 = F_1 + F_2$

$F_4 = F_2 + F_3$

Therefore,

$F_5 = 5$

Thus, Fibonacci is a recursive technique because the function refers to itself using smaller values of n.

39. Explain the recursive algorithm for finding the factorial of a number with an example.

Answer:

The factorial of a positive integer n is represented by n! and is defined as:

$n! = 1 \times 2 \times 3 \times ... \times (n-1) \times n$

The recursive definition is:

If n = 0,

$n! = 1$

If n > 0,

$n! = n \times (n-1)!$

Here, n = 0 is the **base value**. The value for a larger n is defined using a smaller value of n, which moves towards the base value.

Recursive Procedure

Procedure Factorial(Fact, n)

1. If n = 0 then
2. Set Fact := 1 and return
3. Call Factorial(Fact, n-1)
4. Set Fact := n × Fact
5. Return

Example

To find 4!:

$4! = 4 \times 3!$

$3! = 3 \times 2!$

$2! = 2 \times 1!$

$1! = 1 \times 0!$

$0! = 1$

Therefore,

$4! = 4 \times 3 \times 2 \times 1$

$= 24$

Thus, factorial is a recursive technique because the factorial function calls itself with a smaller value of n.

40. Explain the Tower of Hanoi problem with rules and algorithm.

Answer:

The **Tower of Hanoi** is a mathematical puzzle consisting of three pegs:

- **Source (A)**
- **Auxiliary (B)**
- **Destination (C)**

A set of disks of different sizes is placed on the source peg. The largest disk is at the bottom and the smallest disk is at the top.

The objective is to transfer the entire tower from the **Source peg A to Destination peg C**, using the Auxiliary peg B.

Rules

1. Only one disk can be transferred at a time.
2. Only the uppermost disk of a peg can be moved.
3. A larger disk must never be placed on a smaller disk.

Recursive Steps

For N disks:

1. Move the top N-1 disks from peg A to peg B using C as auxiliary.
2. Move the bottom disk from peg A to peg C.
3. Move the N-1 disks from peg B to peg C using A as auxiliary.

Algorithm

```
TOH(n, Sour, Aux, Des)
If (n = 1)
   Write ("Move Disk", n, "from", Sour, "to", Des)
Else
   TOH(n-1, Sour, Des, Aux)
   Write ("Move Disk", n, "from", Sour, "to", Des)
   TOH(n-1, Aux, Sour, Des)
END
```

For example, when n = 3, the algorithm recursively moves the smaller disks first, then moves the largest disk to the destination, and finally moves the smaller disks onto it.

Thus, Tower of Hanoi is an important example of the use of **recursion**.

DATA STRUCTURES USING C++
MODULE 2

I. Objective Type Question: (1 Mark)

1. What is the principle followed by a stack?
A) FIFO
B) LIFO
C) Random order
D) Priority order
Answer: B) LIFO

2. What is the notation of the expression a+b*c?
A) Infix
B) Postfix
C) Prefix
D) Reverse Polish
Answer: A) Infix

3. What is a key feature of postfix expression evaluation?
A) Requires parentheses
B) Requires operator precedence
C) Does not require parentheses
D) Requires sorting
Answer: C) Does not require parentheses

4. What is the basis for processing elements in a priority queue?
A) Insertion order
B) Random order
C) Priority
D) Position
Answer: C) Priority

5. What is stack underflow?
A) Inserting into full stack
B) Deleting from empty stack
C) Inserting the top element
D) Traversing stack
Answer: B) Deleting from empty stack

6. What is the notation of the expression +a*bc?
A)Infix
B)Prefix
C)Postfix
D) Reverse Polish
Answer: B) Prefix

7. Which field stores the elements in a linked representation of a stack?
A) INFO fields
B) TOP fields

C) LINK fields
D) NULL fields
Answer: A) INFO fields

8. Which structures are used to represent a stack?
A) Array only
B) Linked list only
C) Both array and linked list
D) Tree only
Answer: C) Both array and linked list

9. What is the function of the "peek" operation in a stack?
A) Deletes top element
B) Adds element
C) Displays the top element without removing it
D) Sorts elements
Answer: C) Displays the top element without removing it

10. What is the order in which elements are removed from a queue?
A) FIFO
B) LIFO
C) Random order
D) Priority order
Answer: A) FIFO

II. Objective Type Question: (1 Mark)

1. What happens when an element is pushed onto a stack?
A) An element is added to the top
B) An element is removed from the top
C) An element is added to the bottom
D) The stack becomes empty
Answer: A) An element is added to the top

2. What is meant by adding an element to a queue?
A) Pop
B) Push
C) Enqueue
D) Dequeue
Answer: C) Enqueue

3. What is the main idea of the TOP pointer in a stack?
A) It identifies the top element of the stack
B) It identifies the first queue element
C) It identifies the rear of a queue
D) It identifies the bottom of a queue
Answer: A) It identifies the top element of the stack

4. Why a circular queue can reuse vacant positions.
A) The last position connects to the first
B) The first position is always deleted
C) The queue never performs deletion

D) The queue stores elements randomly
Answer: A) The last position connects to the first

5. What is meant by removing an element from the top of a stack?
A) Push operation
B) Pop operation
C) Enqueue operation
D) Dequeue operation
Answer: B) Pop operation

6. How does a linked representation store elements in a stack?
A) Using connected nodes
B) Using only array positions
C) Using tree branches
D) Using queue positions
Answer: A) Using connected nodes

7. How does a linked list stack differ from an array stack in memory usage?
A) It can grow dynamically
B) It always has fixed size
C) It cannot store elements
D) It uses no memory
Answer: A) It can grow dynamically

8. Where can insertion be performed in a double-ended queue (deque)?
A) At the front or rear
B) Only at the front
C) Only at the rear
D) Only at the middle
Answer: A) At the front or rear

9. What is the main idea of the REAR position in a queue?
A) It identifies where insertion occurs
B) It identifies where deletion occurs
C) It identifies the top of a stack
D) It identifies the bottom of a stack
Answer: A) It identifies where insertion occurs

10. What happens when an element is deleted from a queue?
A) FRONT moves forward
B) REAR moves forward
C) TOP moves forward
D) FRONT becomes the last position
Answer: A) FRONT moves forward

III. Short Answer Questions: (3 Marks)

1. What is meant by a queue, and what are the key operations?

Answer:

A queue is a linear data structure that follows the FIFO (First In First Out) principle, where elements are inserted at the rear and removed from the front using FRONT and REAR pointers.

Key Operations:

1. Enqueue: Adds an element at the rear of the queue
2. Dequeue: Removes an element from the front of the queue

It ensures that elements are processed in the same order in which they are inserted.

2. Explain any six applications of the queue data structure.

Answer:

1. CPU Scheduling:
Used to manage processes in First Come First Serve (FCFS) order.

2. Resource Sharing:
Maintains waiting lists for shared resources like CPU, disk, or printer.

3. I/O Buffering:
Handles asynchronous data transfer between input and output devices.

4. Printer Spooling:
Stores print jobs and processes them sequentially.

5. Network Communication:
Manages data packets in routers and switches.

6. Inter-process Communication:
Supports communication through pipes and message queues.

\

3. Explain any three applications of the stack data structure.

Answer:

1. Expression Evaluation:

It supports the evaluation of arithmetic expressions by storing operands and operators, allowing the program to process them in the correct order when multiple operations are involved.

2. Backtracking:

It supports backtracking algorithms by storing previous states, allowing the program to return to earlier steps and try alternative solutions when the current path fails.

3. Delimiter Checking:

It supports delimiter checking by storing opening symbols, allowing the program to match them with closing symbols and detect errors when they are unbalanced.\

4. Summarize the concepts of a priority queue and a deque in data structures.

Answer:

Priority Queue:

It organizes elements based on priority, allowing the program to process higher priority elements before lower priority ones, while elements with the same priority are processed in the order they were inserted.

Deque (Double-Ended Queue):
It allows insertion and deletion of elements at both ends, enabling flexible operations depending on the requirement.

Types of Deque:
1. Input Restricted Deque:
It allows insertion at only one end, while deletion can be performed from both ends when required.
2. Output Restricted Deque:
It allows deletion at only one end, while insertion can be performed at both ends when needed.

5. Explain the algorithm for the PUSH operation in a stack.
Answer:
Algorithm: PUSH(S, TOP, X)
This operation inserts a new element **X** into the top of a stack represented by an array **S**. **TOP** indicates the current top position of the stack. Initially, **TOP = -1**.

1. **Check Overflow Condition:**
   If $TOP \geq N - 1$, the stack is full. Display **"OVERFLOW"** and stop the operation.
2. **Increment TOP:**
   Increase the value of **TOP** by 1.
   TOP = TOP + 1
3. **Insert Element:**
   Place the new element **X** at the top position of the stack.
   S[TOP] = X
4. **Finish Operation:**
   The PUSH operation is completed successfully after inserting the element into the stack.

6. Explain the algorithm for the POP operation in a stack.
Answer:
Algorithm: POP(S, TOP)
This function removes the top element from a stack represented by an array **S** and returns the removed element. **TOP** indicates the current top position of the stack. Initially, **TOP = -1**.

1. **Check Underflow Condition:**
   If **TOP = -1**, the stack is empty. Display **"STACK UNDERFLOW"** and stop the operation.
2. **Store the Top Element:**
   Assign the top element to a variable.
   X = S[TOP]
3. **Decrement TOP pointer:**
   Reduce the value of **TOP** by 1.
   TOP = TOP - 1
4. **Return Element:**
   Return the removed element.
   Return X

7. Explain the PUSH and POP operations on a stack with an example.
Answer:
A stack is a linear data structure that follows the **LIFO (Last In First Out)** principle. The two main operations are PUSH and POP.

- **PUSH:** It is used to insert an element at the top of the stack.
- **POP:** It is used to remove the top element from the stack.

Example:
Initially, the stack is empty.
- PUSH(10) → Stack: 10
- PUSH(20) → Stack: 10, 20
- PUSH(30) → Stack: 10, 20, 30
- POP() → removes 30 → Stack: 10, 20
- POP() → removes 20 → Stack: 10

Final Stack: 10

8. Explain the process of infix to postfix conversion using a stack with an example.
Answer:
Infix expression conversion uses a stack to temporarily store operators and parentheses while scanning the expression. The stack helps in maintaining the correct order of operators based on their precedence and generates the equivalent postfix expression.

Example: A + B * C

Conversion using Stack:

Symbol	**Stack**	**Postfix**
A		**A**
+	+	**A**
B	+	**AB**
*	+*	**AB**
C	+*	**ABC**
END		**ABC*+**

Final Answer:
The postfix form of **A + B * C** is **ABC*+**.

9. How does stack representation relate to its operations, such as push and pop?
Answer:

A stack can be represented using an **array or a linked list**, and both follow the **LIFO (Last In First Out)** principle.

- In **array representation**, elements are stored in a fixed-size array, and the **top** variable keeps track of the last inserted element.
- In **linked list representation**, elements are stored as nodes, and the **top pointer** points to the first node.

Relation to Operations:

- **Push:** Inserts an element at the top
- **Pop:** Removes the element from the top
  Both operations are performed similarly in both representations, but arrays have fixed size, whereas linked lists are dynamic.

Example:
Push(10), Push(20), Pop() → Stack becomes 10

10. Explain the evaluation of a postfix expression using a stack with an example.
Answer:
In postfix evaluation, operands are pushed into the stack, and operators perform operations on the top two elements.

Example: 52+83-*

Symbol	Operation	Stack
5	**Push**	**5**
2	**Push**	**5, 2**
+	**5+2=7**	**7**
8	**Push**	**7, 8**
3	**Push**	**7, 8, 3**
-	**8−3=5**	**7, 5**
*	**7×5=35**	**35**

Final Answer: 35

IV. Long Answer: (5 Marks)

1. Explain the process of converting an infix expression into a postfix expression using a stack.
Answer:
Infix expression conversion uses a stack to temporarily store operators and parentheses.
Steps:

1. Create an empty stack and an output list.
2. Scan the infix expression from left to right.
3. If an operand is encountered, add it directly to the output.

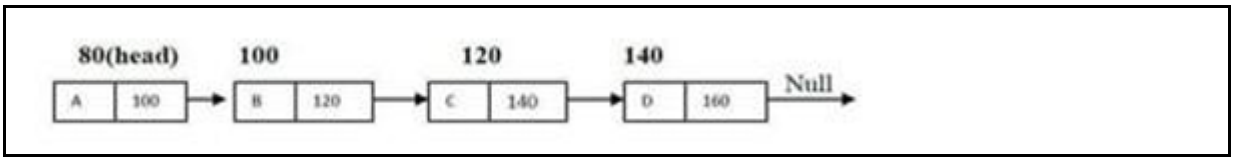
4. If a left parenthesis '(' is encountered, push it onto the stack.
5. If an operator is encountered, pop operators having higher or equal precedence and add them to the output. Then push the current operator.
6. If a right parenthesis) is encountered, pop operators until (is found. Remove the (.
7. After scanning the complete expression, pop all remaining operators from the stack and add them to the output.

2. Illustrate the linked list representation of a queue.
Answer:
In linked representation, a queue is implemented using nodes. Each node contains two fields:

- **INFO** – stores the data element.
- NEXT – stores the address of the next node.

Two pointers are used:

- **FRONT** – points to the first node.
- REAR – points to the last node.



Here, A is the first element, and D is the last element.
Working

1. Insertion (Enqueue): A new node is added at the REAR.
2. Deletion (Dequeue): A node is removed from the FRONT.
3. When the queue is empty, both FRONT and REAR are NULL.
4. The linked representation can grow dynamically as new nodes are created.

Thus, a linked queue does not require a fixed-size array and can use memory dynamically.

3. What is meant by a postfix expression? Find the value of the following postfix expression: 5 6 2 + * 12 4 / - using a stack.
Answer:
A postfix expression is an expression in which operators are written after their operands.
Example:
Postfix: A B +

Evaluation using a stack

Symbol	Action	Stack
5	Push	[5]
6	Push	[5, 6]
2	Push	[5, 6, 2]
+	6 + 2 = 8	[5, 8]
*	5 × 8 = 40	[40]
12	Push	[40, 12]
4	Push	[40, 12, 4]
/	12 ÷ 4 = 3	[40, 3]

-	40 - 3 = 37	[37]

Final Answer = 37

4. Illustrate the evaluation of the following postfix expression: 2 3 1 * + 9 - using a stack.
Answer:

Symbol	Action	Stack (Top at Right)
2	Operand → Push	[2]
3	Operand → Push	[2, 3]
1	Operand → Push	[2, 3, 1]
*	Pop(1, 3) → 3 × 1 = 3 → Push	[2, 3]
+	Pop(3, 2) → 2 + 3 = 5 → Push	[5]
9	Operand → Push	[5, 9]
–	Pop(9, 5) → 5 – 9 = -4 → Push	[-4]

Postfix Expression Evaluation:

Final Output = -4

5. Explain how an element is inserted into a circular queue with a suitable algorithm.
Answer:
Insertion in a circular queue is called **Enqueue**. The element is inserted at the **REAR**.
Algorithm:
Procedure CQINSERT(F, R, Q, MAX, Y):
1. If (REAR + 1) % MAX = FRONT, display OVERFLOW and exit.
2. If FRONT = -1 AND REAR = -1, set FRONT = REAR = 0.
3. Else if REAR = MAX - 1, set REAR = 0.
4. Else set REAR = (REAR + 1) % MAX.
5. Set Q[REAR] = Y.
6. Exit.

Thus, the element is inserted at the **REAR**, and the circular arrangement allows available positions to be reused.

6. Explain how an element is deleted from a circular queue with a suitable algorithm.
Answer:
Deletion in a circular queue is called **Dequeue**. The element is deleted from the **FRONT**.
Algorithm:
FUNCTION CQDELETE(F, R, Q, MAX):
1. If FRONT = -1, display **UNDERFLOW** and exit.
2. Set VAL = Q[FRONT].
3. If FRONT = REAR, set FRONT = REAR = -1.
4. Else if FRONT = MAX - 1, set FRONT = 0.
5. Else set FRONT = (FRONT + 1) % MAX.
6. Exit.

Thus, the element is deleted from the **FRONT**, and the FRONT moves circularly to the next position.

7. Illustrate the insertion and deletion operations in a linked queue with suitable algorithms.
Answer:
A linked queue uses FRONT and REAR pointers. Insertion is performed at the REAR, while deletion is performed from the FRONT.

1. Insertion Operation – LQINSERT
Procedure LQINSERT(INFO, NEXT, ITEM, REAR, FRONT)

1. N = new node
2. If N == NULL, write "Overflow" and exit.
3. Set N → INFO = ITEM.
4. If FRONT == NULL, set FRONT = REAR = N.
5. Otherwise, set REAR → NEXT = N.
6. Set REAR = N.
7. Set N → NEXT = NULL.
8. Finished.

2. Deletion Operation – LQDELETE
Procedure LQDELETE(INFO, NEXT, FRONT, REAR, ITEM, TEMP)

1. If FRONT == NULL, write **"Underflow"** and exit.
2. Set TEMP = FRONT.
3. Set ITEM = FRONT → INFO.
4. Set FRONT = TEMP → NEXT.
5. If FRONT == NULL, set REAR = NULL.
6. Delete TEMP.
7. Finished

Thus, in a linked queue, insertion takes place at the REAR and deletion takes place at the FRONT, following the FIFO (First In, First Out) principle.

8. How does a stack relate to function calls in programming? Explain with a suitable example.
Answer:
A stack is used to manage function calls and returns in a program. It follows the LIFO (Last In, First Out) principle.
Each stack frame stores:

- Return address
- Parameters
- Local variables

Example:

```
main()
{
   A();
}
A()
```

```
{
   B();
}
B()
{
   printf("B Running");
}
```

- When main() calls A(), its information is pushed onto the stack.
- When A() calls B(), B() is pushed onto the stack.
- When B() finishes, it is removed first.
- Then A() returns, followed by main().

Thus, the stack maintains the order of function calls and returns.

9. Illustrate the different types of queues with examples.
Answer:
The main types of queues are Simple Queue, Circular Queue, Double-Ended Queue (Deque), and Priority Queue.

1. Simple Queue
In a simple queue, insertion is performed at the REAR, and deletion is performed at the FRONT. It follows FIFO.
Example:
Initial:
FRONT → [10] → [20] → [30] ← REAR
Insert 40:
FRONT → [10] → [20] → [30] → [40] ← REAR
Delete 10:
FRONT → [20] → [30] → [40] ← REAR

2. Circular Queue
In a circular queue, the last position is connected back to the first position, allowing unused spaces to be reused.
Example:
Initial:
FRONT → [10] → [20] → [30] ← REAR
Delete 10:
FRONT → [20] → [30]
Insert 40 in the empty position:
FRONT → [20] → [30] → [40] ← REAR

Thus, a circular queue reuses the empty space created by deletion.

3. Double-Ended Queue (Deque)
A deque allows insertion and deletion at both the FRONT and REAR.
Example:
Initial:
FRONT ↔ [10] ↔ [20] ↔ [30] ↔ REAR

Insert 5 at FRONT:
FRONT ↔ [5] ↔ [10] ↔ [20] ↔ [30] ↔ REAR
Insert 40 at REAR:
FRONT ↔ [5] ↔ [10] ↔ [20] ↔ [30] ↔ [40] ↔ REAR

4. Priority Queue
In a priority queue, elements are processed according to their priority, rather than simply their arrival order.
Example:
Initial:
FRONT → [10,P3] → [20,P1] → [30,P2] ← REAR
Here, P1 has the highest priority.
Processing order:
20 → 30 → 10

Thus, different types of queues are used based on how elements are inserted, deleted, and processed.

10. What is the main idea of a double-ended queue (Deque)? Explain its insertion and deletion operations with examples.
Answer:
A Double-Ended Queue (Deque) is a queue in which insertion and deletion can be performed at both the FRONT and REAR ends.
Types of Deque
1. Input-Restricted Deque
- Insertion is allowed only at **the REAR.**
- Deletion is allowed at **both FRONT and REAR**.

Example:
Initial:
FRONT → [10] → [20] → [30] ← REAR
Insert 40 at REAR:
FRONT → [10] → [20] → [30] → [40] ← REAR
Delete 10 from FRONT:
FRONT → [20] → [30] → [40] ← REAR
Delete 40 from REAR:
FRONT → [20] → [30] ← REAR

2. Output-Restricted Deque
- Insertion is allowed at **both FRONT and REAR**.
- Deletion is allowed only at **FRONT**.

Example:
Initial:
FRONT → [10] → [20] → [30] ← REAR
Insert 5 at FRONT:
FRONT → [5] → [10] → [20] → [30] ← REAR
Insert 40 at REAR:

FRONT → [5] → [10] → [20] → [30] → [40] ← REAR
Delete 5 from FRONT:
FRONT → [10] → [20] → [30] → [40] ← REAR

Thus, a deque allows insertion and deletion at both ends, while input-restricted and output-restricted deques limit one of these operations.

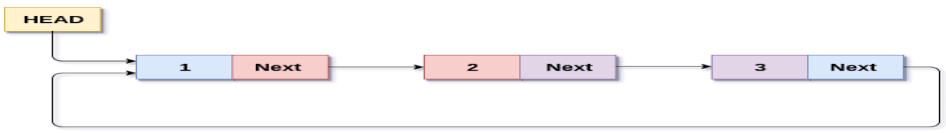
MODULE III

Objective Type Questions (1 Mark)

81	Which of the following explains the role of a node in a doubly linked list? A. A node that stores data and connects to both previous and next nodes B. A node that connects in one direction only C. A node that stores only data D. A node used only at the end of the list Answer: A node that stores data and connects to both previous and next nodes
82	What is the main idea of a list in data structures? A. It stores items without any link B. It stores data in a tree format C. It stores items one after another in order D. It stores data in a random way Answer: C. It stores items one after another in order
83	How does a circular linked list differ from other linked lists? A. It ends with a NULL link B. It has only one node C. It has no data field D. The last node connects back to the first node Answer: D. The last node connects back to the first node
84	Which of the following explains the role of the link field in a linked list node? A. Link field stores the total size of the list B. Link field points to the next node in the list C. Link field stores the data value of the node D. Link field stores the name of the list Answer: B. Link field points to the next node in the list
85	How does a linked list work as a dynamic data structure? A. It allows adding or removing data items at any time

	B. It stores data in arrays C. It arranges data in sequence only D. It uses fixed size memory Answer: A. It allows adding or removing data items at any time
86	What is the purpose of realloc()? A) To release memory B) To initialize memory C) To resize previously allocated memory D) To create a linked list Answer: C) To resize previously allocated memory
87	Which operation should be performed when a dynamically allocated linked-list node is deleted? A) Allocate it again B) Use free() to release its memory C) Use realloc() with zero always D) Use calloc() Answer: B) Use free() to release its memory
88	In a doubly linked list, why does each node contain two pointers along with the data? A) To store two different data values B) To point to the previous and next nodes C) To store the address of the first node only D) To point to two different linked lists Answer: B) To point to the previous and next nodes
89	Which statement correctly allocates memory for 10 nodes of type struct node? A) malloc(10); B) malloc(10 * sizeof(struct node)); C) calloc(sizeof(struct node)); D) alloc(10 * struct node); Answer: B) malloc(10 * sizeof(struct node));
90	If the first node of a circular linked list is deleted, which node should become the new first node?

	A) Last node B) Second node C) Deleted node D) NULL always Answer: B) Second node
91	You need to store 50 user-entered numbers in C++ using dynamic memory without initializing values. Which function will you use? A) malloc() B) calloc() C) realloc() D) free() Answer: A) malloc()
92	Identify the situation that occurs when an attempt is made to remove an element from a linked list whose START pointer is NULL. A) Overflow B) Underflow C) Memory allocation D) Traversal Answer: B) Underflow
93	A singly linked list contains 10 → 20 → 30 → NULL. If node 20 is deleted, where should node 10 point? A) NULL B) 20 C) 30 D) Head Answer: C) 30
94	A student wants to traverse 10 → 20 → 30 → NULL. After visiting 30, what should the traversal do? A) Move to 10 B) Move to 20 C) Stop D) Delete 30 **Answer: C**
95	A system requires insertion and deletion operations at both the front and rear ends. Which linear data structure would be most suitable here?

	A) Stack B) Queue C) Double-Ended Queue (Deque) D) Array Answer: C) Double-Ended Queue (Deque)
96	If dynamically allocated array elements are required to have an initial value of zero, which function should be used? A) malloc() B) calloc() C) realloc() D) free() Answer: B) calloc()
97	A music player needs to move from the current song to both the previous and next songs. Which list structure best supports this requirement? A) Singly linked list B) Doubly linked list C) Header linked list D) Linear array Answer: B .Doubly linked list
98	10 → 20 → 30 → NULL After deleting the first node, what will head point to? A) 10 B) 20 C) 30 D) NULL Answer: B) 20
99	Consider a circular linked list:  What should the next pointer of node 3 contain? A) NULL B) Address of Node 1 C) Address of Node 2 D) Address of Node 3 Answer: B) Address of Node 1

100	If head == NULL before traversing a singly linked list, what does it indicate? A) The list contains one node B) The list is empty C) The list is full D) The list contains multiple nodes Answer: B) The list is empty
Short Answer Questions (3 Marks)	
101	How do you implement an algorithm to search for a specific element in a linked list? ANSWER: SEARCH(INFO, NEXT, HEAD, ITEM, LOCATION) This algorithm finds the location LOCATION of the node where ITEM first appears, or sets LOCATION=NULL. 1. Set temp=head 2. While(temp-> next!=NULL) Begin 3. If ITEM=temp->info 4. Set LOCATION=temp and exit 5. Else 6. Set temp=temp->next 7. End if End while 8. Set LOCATION=NULL // search is unsuccessful 9. Exit
102	Demonstrate the procedure to sequentially access all elements of the linked list. ÀNSWER: Procedure TRAVERSE (LIST, TEMP,INFO, HEAD,NEXT) Let LIST be a linked list in memory. The variable TEMP points to the node which is currently being processed. 1. Set Temp=head 2. While(temp! =NULL) 3. Begin 4. Display temp->info

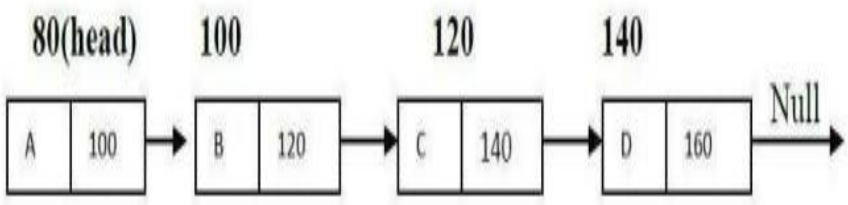
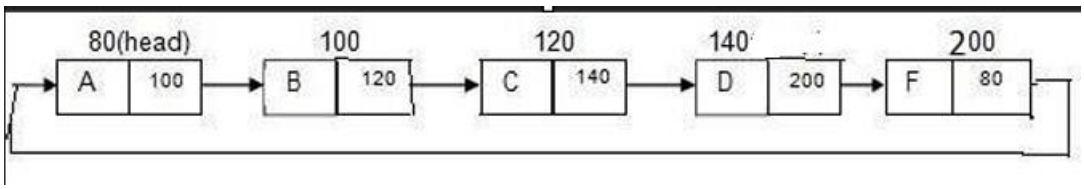
	End 5. Set temp=temp->next 6. Finished
103	Demonstrate the procedure for deleting the head node of a Linked List. ANSWER: Procedure Delete_Head () :Temp is a temporary variable where the head node is stored temporarily before deletion takes place. 1. temp=head 2. Head=head->next 3. Delete temp 4. Exit
104	How do you insert a node at a middle position in a singly linked list? ANSWER: Procedure insert_middle(n) :Here n represents newnode. After is a node where the new node is to be inserted after it. 1. n->next=after->next 2. After-> next=n 3. Exit
105	Demonstrate the procedure for deleting a node from a specified position in a singly linked list. ANSWER: Procedure Delete_middle(n) : Temp is a temporary variable . After is a node where the node to be deleted after that. 1. Temp=after→next 2. After→next=after→next→next 3. Delete temp 4. Exit
106	A linked list shows increasing memory usage after node deletion. Identify the cause and suggest a suitable solution. ANSWER: **Cause:** When a node is deleted from a linked list, the memory occupied by that node may become unused. If this memory is not returned to the free storage list, it results in increasing memory usage.

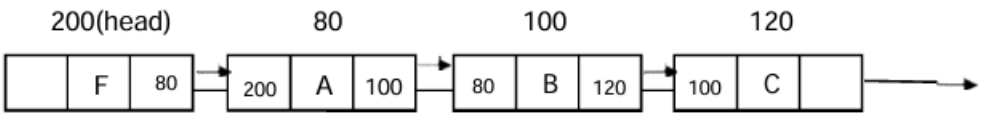
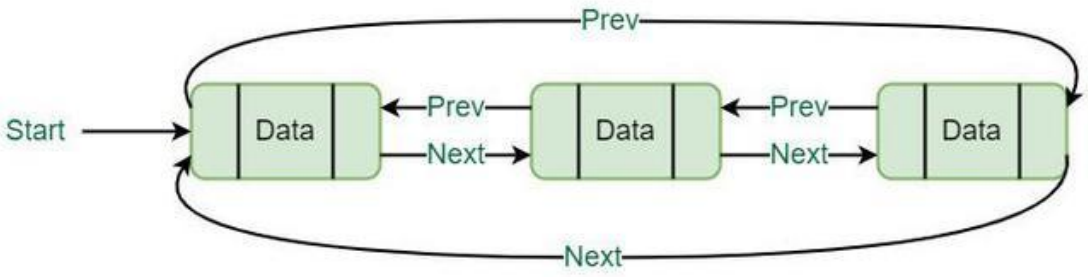
	Solution: The unused memory should be reclaimed through **garbage collection**. Garbage collection identifies memory cells that are no longer in use and adds them to the **free storage list (free pool)** so that they can be reused later.
107	Apply the linked-list traversal procedure to check for the presence of a specified node. ANSWER: SEARCH(INFO, NEXT, HEAD, ITEM, LOCATION) This algorithm finds the location LOCATION of the node where ITEM first appears, or sets LOCATION=NULL. 1. Set temp=head 2. While(temp-> next!=NULL) Begin 3. If ITEM=temp->info 4. Set LOCATION=temp and exit 5. Else 6. Set temp=temp->next 7. End if End while 8. Set LOCATION=NULL // search is unsuccessful 9. Exit
108	How can a new node be added before the first node in a singly linked list? Explain the procedure. ANSWER: Procedure Insert_Head(n) : This algorithm inserts a new node called n to the beginning of a linked list. Head denote the starting node of the linked list . 1. n->next=head 2. Head=n 3. Exit
109	Given a singly linked list 15 → 25 → 35 → NULL, demonstrate the insertion of 20 between 15 and 25 by showing the required pointer changes. ANSWER: Procedure insert_middle(n) : Here:

	• n = new node containing 20 • after = node containing 15 1. n->next=after->next 2. after-> next=n 3. Exit
110	Given a singly linked list 10 → 20 → 30 → 40 → NULL, demonstrate the deletion of node 30 and show the final link structure. ANSWER: For deleting **30**: Let after = node **20** **Pointer changes** 1. Temp = after→next 2. After→next = after→next→next 3. Delete temp **Final list:** 10 → 20 → 40 → NULL
Long Answer Questions (5 Marks)	

111	Describe different dynamic memory allocation functions.
	ANSWER:
	❖ malloc():- Allocate request size of bytes & return a pointer to the first byte of the allocated space. And contains garbage values.
	Syntax:
	ptr = (cast-type*) malloc(byte-size);
	Example:
	ptr = (int*) malloc(100 * sizeof(int));
	❖ calloc():- Allocate space for an array of elements, initialize them to zero and returns a pointer to the first byte of allocated space.
	Syntax:

	ptr = (cast-type*)calloc(n, element-size);
	
	Here, n is the no. of elements and element-size is the size of each element.
	
	Example:
	
	ptr = (int*) calloc(25, sizeof(int));
	
	❖ realloc():- Modify the size of previously allocated space.
	
	Syntax:
	
	ptr = realloc(ptr, newSize);
	
	where ptr is reallocated with new size 'newSize'.
	
	❖ free():- Free the previously allocated space.
	
	Syntax:
	
	free(ptr);

112	Explain a linked list and describe its types with suitable diagrams. ANSWER: A linked list is a non-sequential collection of data items. For every data item in the linked list, there is an associated pointer that gives the memory location of the next data item in the linked list. Types of linked list: **Singly linked list** : A singly linked list is the one in which all nodes are linked together in some sequential manner.  **Circular linked list:** It is a singly linked list in which the link field of the last node contains the address of the first node of the list.  **Doubly Linked List:** A doubly linked list is a one which help in accessing both the successor node and the predecessor node form the given node position.

	 **Doubly circular linked list :** In circular doubly linked list two consecutive elements are linked or connected by previous and next pointer and the last node points to the first node also points to last node by previous pointer. 
113	Explain the concept of insertion operation at the beginning and end of a singly linked list. ANSWER: Procedure Insert_Head(n) : This algorithm inserts a new node called n to the beginning of a linked list. Head denote the starting node of the linked list . 1. n->next=head 2. Head=n 3. Exit Procedure_Insert_end(n) : This algorithm inserts a new node called n at the end of the linked list. Temp is a temporary variable. 1. Temp=head 2. While(temp->next !=NULL) Begin 3. temp=temp->next End 4. temp->next=n 5. n->next=NULL 6. Exit
114	Write an algorithm to search for a specific element in a linked list. ANSWER: SEARCH(INFO, NEXT, HEAD, ITEM, LOCATION)

	This algorithm finds the location LOCATION of the node where ITEM first appears, or sets LOCATION=NULL. 1. Set temp=head 2. While(temp-> next!=NULL) Begin 3. If ITEM=temp->info 4. Set LOCATION=temp and exit 5. Else 6. Set temp=temp->next 7. End if End while 8. Set LOCATION=NULL // search is unsuccessful 9. Exit
115	Explain the concept of deletion operation at the beginning and end of a singly linked list. ANSWER: Procedure Delete_Head () :This algorithm deletes the head node of the linked list. Temp is a temporary variable where the head node is stored temporarily before deletion takes place. 1. temp=head 2. Head=head->next 3. Delete temp 4. Exit Procedure delete_end() : This algorithm deletes the last node of the linked list. Temp is a temporary variable where the head node is stored temporarily before deletion process takes place. 1. Temp=head 2. While(temp→next->next!=NULL) Begin 3. Temp=temp→next End 4. Delete temp→next 5. Temp→next=NULL 6. Exit

116	How do malloc(), calloc(), realloc(), and free() help in managing memory dynamically in C? Explain with their uses.
	
	ANSWER:
	
	❖ malloc():- Allocate request size of bytes & return a pointer to the first byte of the allocated space. And contains garbage values.
	
	Syntax:
	
	ptr = (cast-type*) malloc(byte-size);
	
	Example:
	
	ptr = (int*) malloc(100 * sizeof(int));
	
	❖ calloc():- Allocate space for an array of elements, initialize them to zero and returns a pointer to the first byte of allocated space.
	
	Syntax:
	
	ptr = (cast-type*)calloc(n, element-size);
	
	Here, n is the no. of elements and element-size is the size of each element.
	
	Example:
	
	ptr = (int*) calloc(25, sizeof(int));
	
	❖ realloc():- Modify the size of previously allocated space.
	
	Syntax:
	
	ptr = realloc(ptr, newSize);
	
	where ptr is reallocated with new size 'newSize'.
	
	❖ free():- Free the previously allocated space.
	
	Syntax:
	
	free(ptr);

117	Write the algorithm to traverse the linked list. ÀNSWER: Procedure TRAVERSE (LIST, TEMP,INFO, HEAD,NEXT) Let LIST be a linked list in memory. The variable TEMP points to the node which is currently being processed. 1. Set Temp=head 2. While(temp! =NULL) 3. Begin

<table>
<tr><td></td><td>4. Display temp->info

End

5. Set temp=temp->next

6. Finished</td></tr>
<tr><td>118</td><td>How do you check whether an element is present in a linked list or not?

ANSWER:

SEARCH(INFO, NEXT, HEAD, ITEM, LOCATION)

This algorithm finds the location LOCATION of the node where ITEM first appears, or sets LOCATION=NULL.

1. Set temp=head

2. While(temp-> next!=NULL)

Begin

3. If ITEM=temp->info

4. Set LOCATION=temp and exit

5. Else

6. Set temp=temp->next

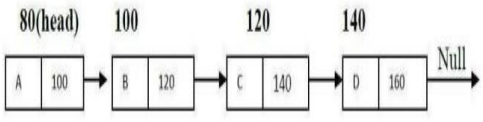
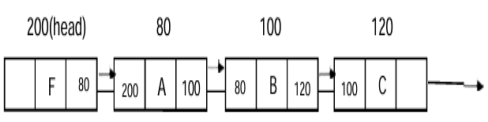
7. End if

End while

8. Set LOCATION=NULL // search is unsuccessful

9. Exit</td></tr>
<tr><td>119</td><td>Analyze and compare singly linked lists and doubly linked lists in terms of structure and operations.

ANSWER:

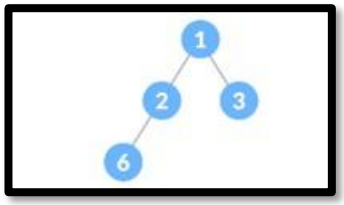
<table>
<tr><th>Singly Linked List</th><th>Doubly Linked List</th></tr>
<tr><td>Each node has data + next pointer.</td><td>Each node has data + previous pointer + next pointer.</td></tr>
<tr><td>Traversal is possible only forward.</td><td>Traversal is possible forward and backward.</td></tr>
<tr><td>Requires less memory.</td><td>Requires more memory because of the extra pointer.</td></tr>
<tr><td>Cannot directly access the previous node.</td><td>Can directly access the previous node.</td></tr>
<tr><td>Easier to implement.</td><td>More complex to implement.</td></tr>
<tr><td> 
</td><td> 
</td></tr>
</table>
</td></tr>
</table>

120	Explain the concept of garbage collection in linked-list memory management. ANSWER: Together with the linked lists in the memory, a special list is maintained which consists of unused memory cells. This list, which has its own pointer, is called the list of available space or the free storage list or the free pool. The operating system of a computer may periodically collect all the deleted space onto the free storage list. Any technique which does this collection is called garbage collection. Garbage collection usually takes place into 2 steps. First the computer runs through all lists, tagging those cells which are currently in use, and then the computer runs through the memory, collecting all untagged space onto free storage list.

MODULE IV

I. Objective Type Questions (1 Mark):

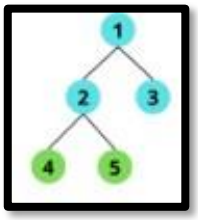
1. Predict the degree of the root node in the binary tree shown below.



A. 0
B. 1
C. 2
D. 3

Answer: C. 2

2. How would you apply preorder traversal to the given binary tree?



A. 1 2 4 5 3

B. 4 2 5 1 3

C. 4 5 2 3 1

D. 2 4 5 1 3

Answer: A.1 2 4 5 3

3. If a binary tree has root = 10, left child = 20, and right child = 30, what is the inorder traversal?

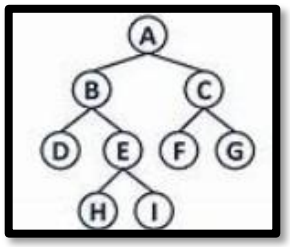
A) 10, 20, 30
B) 20, 10, 30
C) 20, 30, 10
D) 30, 10, 20

Answer: B. 20, 10, 30

4. If the root node is at level 0 and a node is three edges away from the root, what is its level?
   A) 2
   B) 3
   C) 4
   D) 1

Answer: B) 3

5. How many leaf nodes are present in the following binary tree:



A. 1

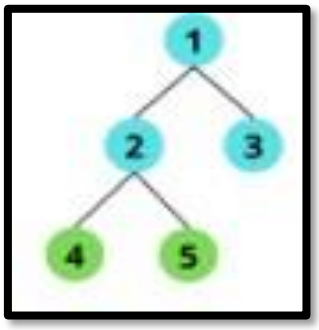
B. 6

C. 4

D. 5

Answer: D.5

6. How would you apply inorder traversal to the given binary tree?



A. 1 2 4 5 3

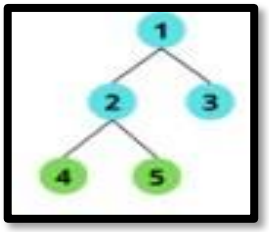
B. 4 2 5 1 3

C. 4 5 2 3 1

D. 2 4 5 1 3

Answer: B. 4 2 5 1 3

7. How would you apply post-order traversal to the given binary tree?



A. 1 2 4 5 3

B. 4 2 5 1 3

C. 4 5 2 3 1

D. 2 4 5 1 3

Answer: C. 4 5 2 3 1

8 How do you search for a node in a Binary Search Tree (BST)?

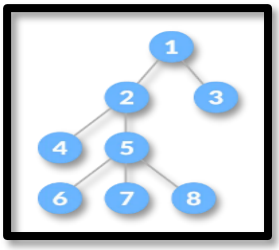
A. Compare with the root; go left if smaller, right if larger.
B. Search every node one by one.
C. Compare only with leaf nodes.
D. Insert the key, then search.

Answer: A. Compare with the root; go left if smaller, right if larger.

9 In a tree, nodes B and C are connected to the same parent. What is the relationship between B and C?
A) Ancestors
B) Parents
C) Siblings
D) Roots
Answer: C) Siblings

10 In a tree, the path from A to D is A → B → C → D. How many edges are in this path?
A) 2
B) 3
C) 4
D) 5
Answer: B) 3

11 In the given tree diagram, identify the leaf nodes.



A. 4 6 7 8 3

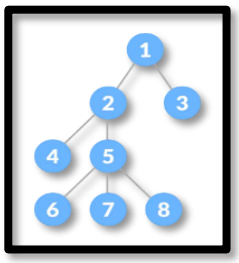
B. 3 8 7 6 4

C. 8 7 6 4 3

D . 6 7 8 3 4

Answer: 4 6 7 8 3

12 In the given tree diagram, identify the root node.



A 1
B 2
C 3
D 4

Answer: A. 1

13 Consider this BST:

50

/ \

30 70

/ \

20 40

If node 30 is deleted, which node can replace it while maintaining the BST property?

A) 20

B) 40

C) 70

D) 50

Answer: B

14 . Identify the visiting order of nodes when preorder traversal is applied on a binary tree.

A. Root, Left Sub-tree, Right Sub-tree

B. Left Sub-tree, Root, Right Sub-tree

C. Left Sub-tree, Right Sub-tree, Root

D. Right Sub-tree, Root, Left Sub-tree

Answer**:** Root, Left Sub-tree, Right Sub-tree

15. A node has three children in a tree. Identify the degree of that node.

A) 1

B) 2

C) 3

D) 4

Answer: C) 3

16 If node A is the root and is connected to B, what is B with respect to A?

A) Parent

B) Ancestor

C) Child

D) Sibling

Answer: C) Child

17 In a binary search tree, 40 is the root and 60 is inserted. Where should 60 be placed?

A) Left of 40

B) Right of 40

C) As the root

D) Outside the tree

Answer: B) Right of 40

18 For the tree 10 → 20 → 30, where each node has only one child, what is the maximum depth?

A) 1

B) 2

C) 3

D) 0

Answer: B) 2

19 In a tree, node X is the parent of Y, and Y is the parent of Z. What is X with respect to Z?

A) Sibling

B) Ancestor

C) Terminal node

D) Child
Answer: B) Ancestor

20 In a binary search tree, 25 is the root and 15 is inserted. Where should 15 be placed?
A) Right of 25
B) Left of 25
C) Above 25
D) As the root
Answer: B) Left of 25

II. Short Answer Questions (3 Marks):

1. Demonstrate the algorithm for preorder traversal.

Answer:
Procedure Preorder(root)
Step 1: If (root !=NULL)
Step 2: Begin
Step 3: Print root-> info
Step 4: Preorder(root-> left)
Step 5: Preorder(root-> right)
Step 6: End
Step 7: finished

2. How would you use an array to represent array in a binary tree in memory? Answer:

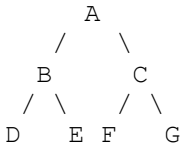
An **array can be used to represent a binary tree in memory** by storing the nodes in specific positions based on their level and relationship with their parent nodes.

For a binary tree, if the **root node is stored at position 1**, then:

- The **left child** of a node at position `k` is stored at position `2k`.
- The **right child** of a node at position `k` is stored at position `2k + 1`.
- The **parent** of a node at position `k` is stored at position `⌊k/2⌋`.

Example:

Consider the following binary tree:



Its array representation is:

Array Position 1 2 3 4 5 6 7

Array Position 1 2 3 4 5 6 7

Node A B C D E F G

Therefore:

- A is stored at TREE[1].
- B is the left child of A, so it is stored at TREE[2].
- C is the right child of A, so it is stored at TREE[3].
- D is the left child of B, so it is stored at TREE[4].
- E is the right child of B, so it is stored at TREE[5].
- F is the left child of C, so it is stored at TREE[6].
- G is the right child of C, so it is stored at TREE[7].

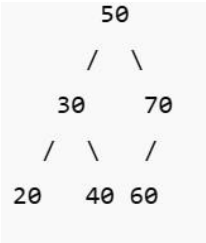
Thus, **array representation provides a simple way to store a binary tree in contiguous memory locations**, especially when the tree is complete or nearly complete.

3. Demonstrate the algorithm for in-order traversal.

Answer:
Algorithm for inorder traversal
Procedure Inorder(root)
Step 1: If (root !=NULL)
Step 2: Begin
Step 3: Inorder(root ->left)
Step 4: Print -> info
Step 5: Inorder(root-> right)
Step 6: End
Step 7: Finished

4. Design a Binary Search Tree for the numbers 50, 30, 70, 20, 40, 60 and determine the degree of each node and identify all sibling nodes?

Answer:



Degrees of nodes:

- 50 → 2
- 30 → 2
- 70 → 1
- 20 → 0
- 40 → 0
- 60 → 0

Siblings:

- 30 & 70
- 20 & 40

5. Demonstrate the algorithm for post-order traversal.

Answer:

Algorithm for post-order traversal:

Procedure Post-order(root)

Step 1: If (root!=NULL)

Step 2: Begin

Step 3: Post-order(root ->left)

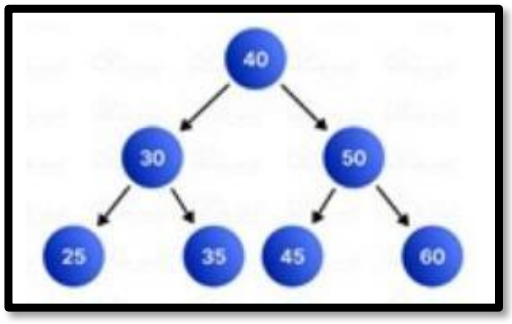
Step 4:Post-order(root ->right)

Step 5:Print root-> info

Step 6:End

Step 7:Finished

6. Find the in-order, pre-order, and post-order traversals of the following binary tree:**:**



Answer:

In-order Traversal:

25, 30, 35, 40, 45, 50, 60

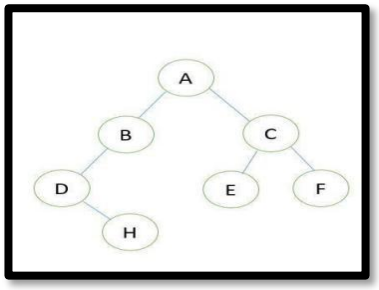
Pre-order Traversal:

40, 30, 25, 35, 50, 45, 60

Post-order Traversal:

25, 35, 30, 45, 60, 50, 40

7. **Demonstrate the array representation of the given binary tree.**



Answer:

1. The root **R** of **T** is stored in **TREE[1]**.
2. If a node **N** occupies **TREE[K]**, then its left child is stored in **TREE[2*K]** and its right child is stored in **TREE[2*K + 1]**.

Step 1: Assign TREE indices

Root **A** → **TREE[1]**
Left child of **A** → **B** → **TREE[2*1] = TREE[2]**
Right child of **A** → **C** → **TREE[2*1 + 1] = TREE[3]**
Left child of **B** → **D** → **TREE[2*2] = TREE[4]**
Right child of **B** → None → **TREE[2*2 + 1] = TREE[5] (empty)**
Left child of **C** → **E** → **TREE[2*3] = TREE[6]**
Right child of **C** → **F** → **TREE[2*3 + 1] = TREE[7]**
Left child of **D** → None → **TREE[2*4] = TREE[8] (empty)**
Right child of **D** → **H** → **TREE[2*4 + 1] = TREE[9]**

8. How do you insert a new roll number into a BST while maintaining the BST property?

A **Binary Search Tree (BST)** maintains the property that:

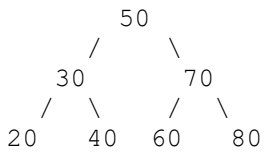
- Values smaller than a node are stored in its **left subtree**.
- Values greater than a node are stored in its **right subtree**.

To insert a new roll number while maintaining the BST property, follow these steps:

1. **Start from the root** of the BST.
2. Compare the new roll number with the current node.
3. If the new roll number is **smaller**, move to the **left child**.
4. If the new roll number is **greater**, move to the **right child**.
5. Continue comparing until an **empty (NULL) position** is found.
6. Insert the new roll number at that position.

Example

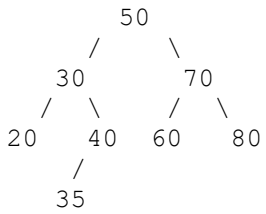
Suppose the BST contains the roll numbers:



If a new roll number **35** is to be inserted:

- Compare `35` with `50` → `35 < 50`, move left.
- Compare `35` with `30` → `35 > 30`, move right.
- Compare `35` with `40` → `35 < 40`, move left.
- The left position of `40` is empty, so insert `35` there.

The resulting BST is:

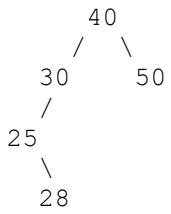


Thus, **the new roll number is inserted at the appropriate position without violating the BST property**.

9. **Construct a Binary Search Tree (BST) for the elements 40, 30, 50, 25, and 28.**

Answer:

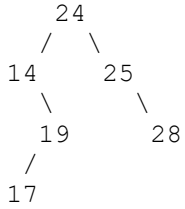
BST:



10. **Construct a Binary Search Tree (BST) for the elements 24, 25, 14, 19, 17, and 28. Write its In-order and Preorder.**

Answer:

BST:



Traversals:

- **In-order (Left–Root–Right):** 14, 17, 19, 24, 25, 28
- **Preorder (Root–Left–Right):** 24, 14, 19, 17, 25, 28

III.Long Answer Questions (5 Marks):

1. How would you determine whether a binary tree is Strictly Binary Tree? Explain with example.

Answer:

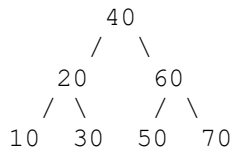
A **Strict Binary Tree** is a binary tree in which **every node has either exactly two children or no children**. In other words, no node is allowed to have only one child.

Steps to verify whether a Binary Tree is a Strict Binary Tree:

1. Start from the **root node** and examine each node.
2. Check the number of children of every node.
3. If a node has **two children**, continue checking the remaining nodes.
4. If a node has **no children**, it is a leaf node, so continue.
5. If any node has **exactly one child**, the tree is **not a Strict Binary Tree**.
6. If all nodes have either **0 or 2 children**, the tree is a **Strict Binary Tree**.

Example:

Consider the following binary tree:

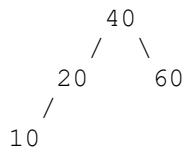


- Node **40** has two children: 20 and 60.
- Node **20** has two children: 10 and 30.
- Node **60** has two children: 50 and 70.
- Nodes **10, 30, 50, and 70** have no children.

Therefore, every node has either **0 or 2 children**.

Hence, the given binary tree is a Strict Binary Tree.

Example of a non-strict binary tree:



Here, node **20** has only **one child (10)**. Therefore, the tree **is not a Strict Binary Tree**.

2 How would you determine whether a binary tree is Complete Binary Tree? Explain with a suitable example.

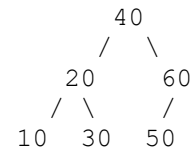
A **Complete Binary Tree** is a binary tree in which **all levels are completely filled except possibly the last level, and the nodes in the last level are filled from left to right**.

To verify whether a given binary tree is a Complete Binary Tree, apply the following steps:

1. Start from the **root node** and examine the tree level by level.
2. Check whether all levels before the last level are completely filled.
3. In the last level, check whether the nodes are arranged **from left to right without any gaps**.
4. A node may have **one child**, but if it has only one child, that child must be the **left child**, and it must occur at the last level.
5. If a vacant position occurs before an existing node at the same level, the tree is **not a Complete Binary Tree**.

Example:

Consider the following binary tree:



- Level 1 contains **40**.
- Level 2 contains **20 and 60**, so it is completely filled.
- At the last level, nodes **10, 30, and 50** are filled from **left to right** without any gap.
- Node **60** has only one child, and that child is the **left child (50)**.

Therefore, all the conditions of a **Complete Binary Tree** are satisfied.

3 Design a Binary Search Tree (BST) for the numbers 30, 20, 40, 15, 18 and determine the level of each node and find the depth of the tree?

Answer:

Insert 30
→ 30 becomes the root node.
30
Insert 20
→ 20 < 30 → goes to the left of 30.

```
  30
 /
20
```

Insert 40
→ 40 > 30 → goes to the right of 30.

```
   30
  / \
 20   40
```

Insert 15
→ 15 < 30 → go left to 20
→ 15 < 20 → go left again → insert here.

```
     30
    / \
  20    40
 /
15
```

Insert 18
→ 18 < 30 → go left to 20
→ 18 < 20 → go left to 15
→ 18 > 15 → go right → insert here.

```
       30
      / \
    20    40
   /
  15
   \
    18
```

Determine the **level of each node**

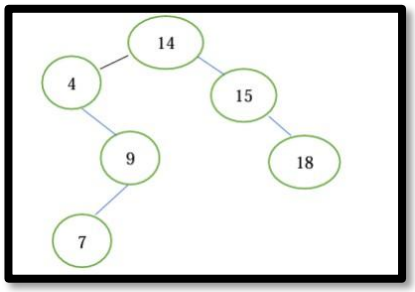
Node	Level	Explanation
30	0	Root
20	1	Left child of 30
40	1	Right child of 30
15	2	Left child of 20
18	3	Right child of 15

- Depth of a tree = the maximum level of any node.
- The deepest node is 18 at level 3.

Depth of the tree = 3

4 Demonstrate how to construct a Binary Search Tree using the values 14, 15, 4, 9, 7, and 18 and then show its In-order, Preorder, and Post-order traversals.

Answer:



INORDER: LEFT-ROOT-RIGHT

4 7 9 14 15 18

PREORDER: ROOT-LEFT-RIGHT

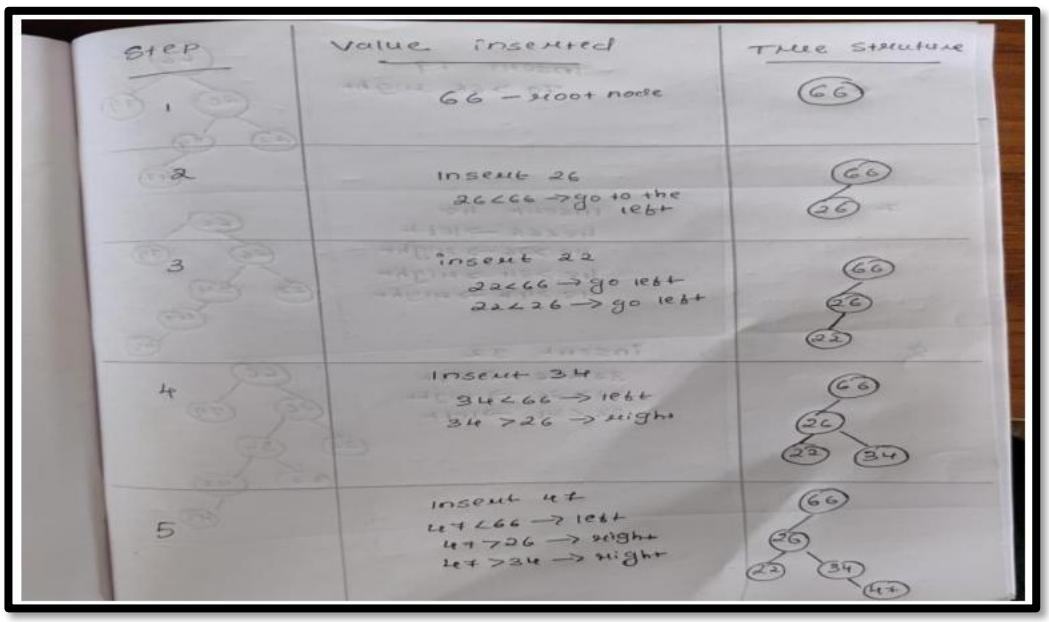
14 4 9 7 15 18

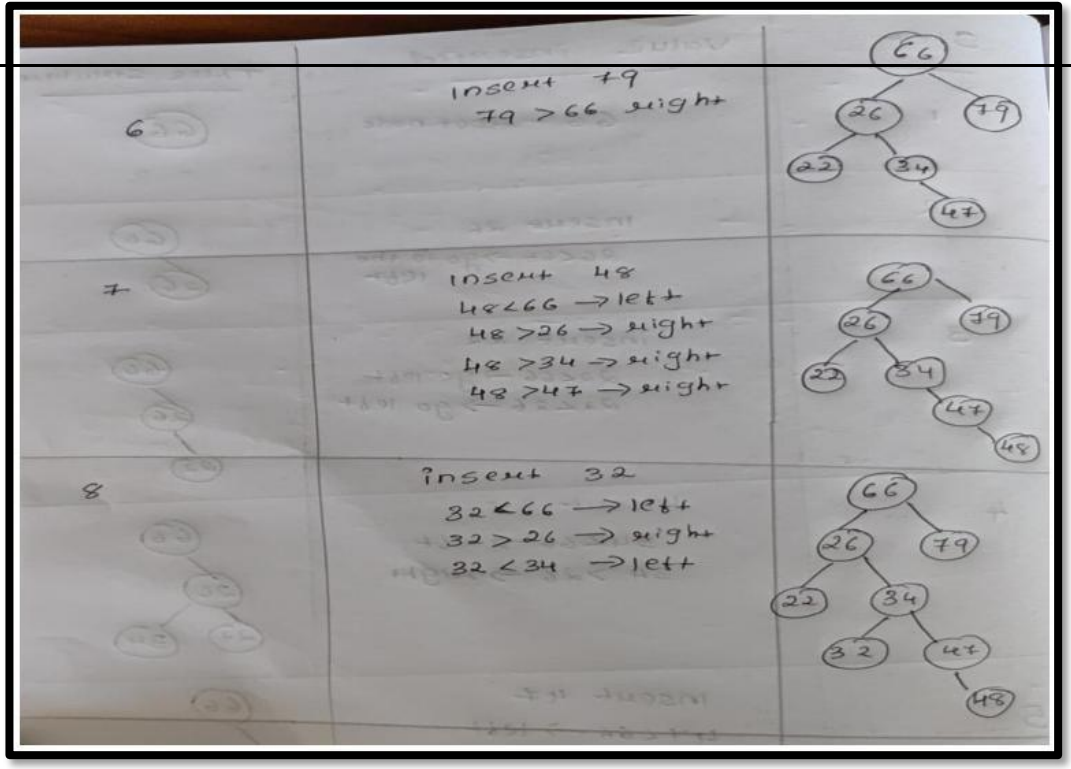
POSTORDER: LEFT-RIGHT-ROOT

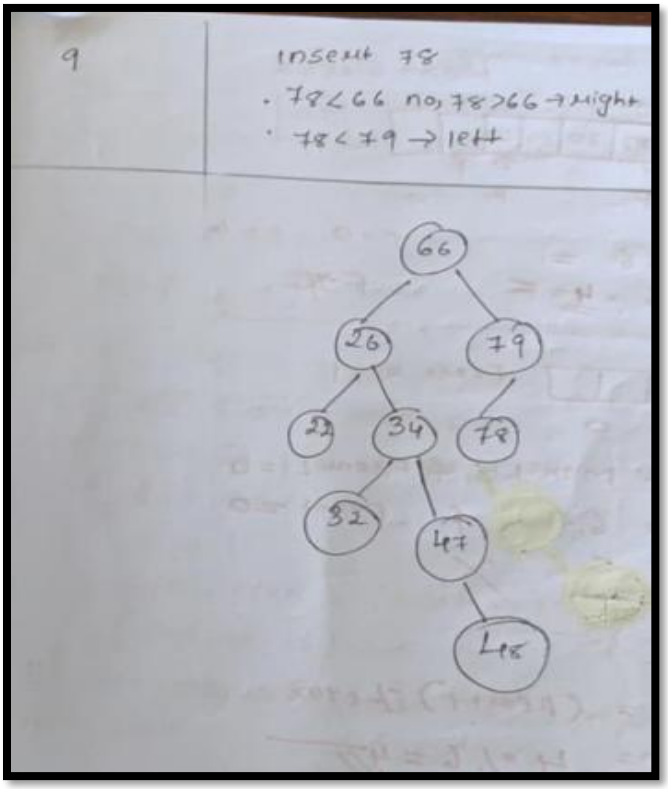
7 9 4 18 15 14

5 Demonstrate how to construct a Binary Search Tree using the values 66, 26,22,34,47,79,48,32,78, and show how the tree would look.

Answer:







6 Design a Binary Search Tree (BST) using the numbers 8, 3, 10, and 1, and then redraw it after deleting a node with no children.

Answer:

Construct the Binary Search Tree (BST)

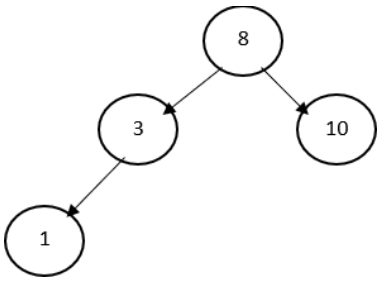
Numbers to insert: **8, 3, 10, 1**

Step-by-step insertion:

1. **8** → becomes the **root**.
2. **3** → less than 8 → goes to the **left** of 8.
3. **10** → greater than 8 → goes to the **right** of 8.

4. **1** → less than 8 → go left to 3 → less than 3 → goes to the **left** of 3.

Resulting BST (Before Deletion)



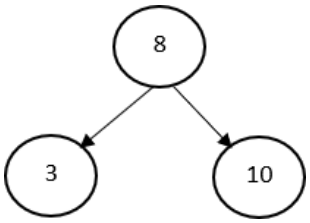
N has no children. Then N is deleted from the tree T by simply replacing the location of N in the parent node P(N) by the NULL pointer

Delete a Node with No Children

In this BST, the **leaf nodes (no children)** are:

- **1**
- **10**

After Deletion of Node 1



7. Apply the Preorder Traversal to traverse a Binary Tree and display its nodes.

Answer:

Preorder Traversal Algorithm (Root–Left–Right):

Algorithm:

```
Procedure Preorder(root)

Step 1: If (root == NULL)
            Return

Step 2: Print root->data

Step 3: Preorder(root->left)

Step 4: Preorder(root->right)

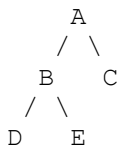
Step 5: End
```

Explanation:

- In **Preorder Traversal**, the **root node is visited first**, followed by the **left subtree**, and then the **right subtree**.
- If the current node is `NULL`, the algorithm stops and returns.
- The process is applied recursively to every node until all nodes are visited.

Traversal Order:
Root → Left → Right (RLR)

Example:



Preorder Traversal: A, B, D, E, C

This algorithm ensures that every node in the binary tree is visited exactly once in **Root–Left–Right** order.

8 Design an array-based representation for storing binary nodes in memory. Illustrate the structure of a node.

Answer:

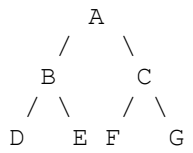
An **array-based representation** can be used to store the nodes of a binary tree in contiguous memory locations. Each node is stored at a specific index in the array based on its position in the tree.

For a binary tree, if the root is stored at position `1`:

- **Left child of node at position** `k` → `TREE[2k]`
- **Right child of node at position** `k` → `TREE[2k+1]`
- **Parent of node at position** `k` → `TREE[⌊k/2⌋]`

Example

Consider the binary tree:



Its array representation is:

```
TREE[1] = A
TREE[2] = B
TREE[3] = C
TREE[4] = D
TREE[5] = E
TREE[6] = F
TREE[7] = G
```

Therefore:

```
Index:    1   2   3   4   5   6   7
         ---------------------------
TREE:     A   B   C   D   E   F   G
```

Structure of a Binary Node

A binary tree node logically contains **data** and links to its **left and right children**:

```
       +-------------------+
       |       DATA        |
       +-------------------+
       |   LEFT |  RIGHT   |
       +-------------------+
```

In an **array representation**, the child links are not stored explicitly. Their positions are calculated using the array indices.

For example, for `TREE[2] = B`:

```
          B
        /   \
TREE[4] TREE[5]
   D       E
```

Thus, the array-based representation provides an efficient method for storing binary trees, particularly **complete and nearly complete binary trees**.

9 How would you apply the In-order Traversal algorithm to visit the nodes of the tree? Illustrate with an example.

Answer:

In-order Traversal Algorithm (Left – Root – Right)

In **In-order Traversal**, the nodes of a binary tree are visited in the following order:

Left Subtree → Root Node → Right Subtree

Algorithm

```
Procedure Inorder(root)

Step 1: If (root == NULL)
            Return

Step 2: Inorder(root->left)

Step 3: Print root->data

Step 4: Inorder(root->right)

Step 5: End
```

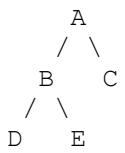
Explanation

- If the current node is **NULL**, the function returns.
- First, recursively visit the **left subtree**.
- Next, visit and print the **root node**.
- Finally, recursively visit the **right subtree**.
- This process continues until all nodes in the tree are visited.

Traversal Order

Left → Root → Right (LNR)

Example



In-order Traversal: D, B, E, A, C

10 How would you apply Post-order Traversal to visit the folders in the required order? Illustrate the traversal with an example.

Answer:

Post-order Traversal Algorithm (Left – Right – Root)

In **Post-order Traversal**, the nodes of a binary tree are visited in the following order:

Left Subtree → Right Subtree → Root Node

Algorithm

```
Procedure Postorder(root)

Step 1: If (root == NULL)
            Return

Step 2: Postorder(root->left)

Step 3: Postorder(root->right)

Step 4: Print root->data

Step 5: End
```

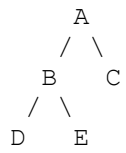
Explanation

- If the current node is **NULL**, the function returns.
- First, recursively visit the **left subtree**.
- Next, recursively visit the **right subtree**.
- Finally, visit and print the **root node**.
- The process continues until every node in the binary tree has been visited.

Traversal Order

Left → Right → Root (LRN)

Example



Post-order Traversal: D, E, B, C, A

MODULE V

I. Objective Type Questions (1 Mark)

1. What approach would you use to check whether an edge is bidirectional in a graph?

A. Check if both (A,B) and (B,A) exist
B. Check for loops
C. Use queue
D. Use DFS only

Answer: A. Check if both (A,B) and (B,A) exist

2. How many passes bubble sort will take to completely sort the list [5,3,2]?

A. 1 pass
B. 2 passes
C. 3 passes
D. No pass needed

Answer: B. 2 Passes

3. During bubble sort, the first pass is performed on 5, 2, 4, 1. Which element reaches the last position?

A. 1
B. 2
C. 4
D. 5

Answer: D. 5

4. What approach would you use to explore one branch completely before visiting another branch?

A. Breadth First Search
B. Depth First Search
C. Binary Search
D. Merge Sort

Answer: B. Depth First Search

5. What approach would you use to sort numbers with high efficiency?

A. Bubble Sort
B. Insertion Sort
C. Merge Sort
D. Linear Search

Answer: C. Merge Sort

6. A maze-solving program explores one path completely before trying another. Which data structure should it use?

A. Queue
B. Stack
C. Matrix
D. Array

Answer: A. Stack

7. What would be the MID value if LOW = 2 and HIGH = 6?

A. 3
B. 4
C. 5
D. 6

Answer: B. 4

8. **What approach would you use to traverse a graph level by level?**
   A. Depth First Search
   B. Breadth First Search
   C. Binary Search
   D. Linear Search

Answer: B. Breadth First Search

9. **What approach would you use to represent a graph when you need to quickly check whether an edge exists between two vertices?**
   A. Linked List Representation
   B. Adjacency Matrix
   C. Stack
   D. Queue

Answer: B. Adjacency Matrix

10. **What would you use to implement Breadth First Search (BFS)?**
   A. Stack
   B. Queue
   C. Array
   D. Linked List

Answer: B. Queue

II. Objective Type Questions (1 Mark)

1. **What approach would you use to search an element in an unsorted array?**
   A. Binary Search
   B. Linear Search
   C. Merge Sort
   D. DFS

   Answer: B. Linear Search

2. **What approach would you use to visit each node?**

   A. Sorting
   B. Searching
   C. Traversal
   D. Deletion

   Answer: C. Traversal

3. **How would you remove an edge from a graph?**
   A. Insertion
   B. Deletion
   C. Traversal
   D. Searching

   Answer: B. Deletion

4. **A sorted list contains 10, 20, 30, 40, 50. To search for 40, the middle element is checked first. Which search technique is being applied?**
   A. Sequential search
   B. Binary search
   C. Linear sorting
   D. Selection

   Answer: B Binary search

5. What would happen if DFS is performed starting from A in the following graph. Which node would be visited immediately after C?
A → [B, C]
B → [D, E]
C → [F]
D, E, F → []

A. B
B. D
C. E
D. F

Answer: D. F

6. Identify the value in the adjacency matrix if there is no edge between two vertices.

A. -1
B. 0
C. 1
D. 2

Answer: B. 0

7. What would happen if the search key is greater than the middle element during Binary Search.

A. The search continues in the left half.
B. The search terminates immediately.
C. The search continues in the right half.
D. The array is sorted again.

Answer: C. The search continues in the right half.

8. In selection sort, the elements are 30, 10, 20. Which element is selected during the first pass for ascending order?

A. 30
B. 20
C. 10
D. 0

Answer: C. 10

9. Identify the sorting technique that places an element in its correct position during each pass.

A. Bubble Sort
B. Quick Sort
C. Insertion Sort
D. Selection Sort

Answer: C. Insertion Sort

10. A graph has 4 vertices. An adjacency matrix is used to represent it. How many rows and columns are required?

A. 4 × 2
B. 2 × 4
C. 4 × 4
D. 3 × 3

Answer: C) 4 × 4

III. Short Answer Questions (3 Marks):

1. Demonstrate the Insertion Sort algorithm.

Answer:

Procedure: Insertion_Sort(A, N)
Step 1: For I = 1 to N do
Step 2: TEMP = A[I]
Step 3: J = I - 1
Step 4: While (J >= 0) and (A[J] > TEMP)
A[J + 1] = A[J]
J = J - 1
[End of while loop]
Step 7: A[J + 1] = TEMP
[End of Step 1 for loop]
Step 8: Exit

2. How would you solve the problem of sorting the numbers 10, 4, 2, and 5 using Bubble Sort? Explain each pass.

Answer:

Array:

0	**1**	**2**	**3**
10	**4**	**2**	**5**

Pass 1:

Compare 10 and 4. Since 10 > 4, swap the elements.

Array:

0	1	2	3
4	10	2	5

Compare 10 and 2. Since 10 > 2, swap the elements.

Array:

0	1	2	3
4	2	10	5

Compare 10 and 5. Since 10 > 5, swap the elements.

Array:

0	1	2	3
4	2	5	10

Pass 2:

Compare 4 and 2. Since 4 > 2, swap the elements.

Array:

0	1	2	3
2	4	5	10

Compare 4 and 5. Since 4 < 5, no swap is required.

Array:

0	1	2	3
2	4	5	10

Pass 3:

Compare 2 and 4. Since 2 < 4, no swap is required.

Array:

0	1	2	3
2	4	5	10

Sorted Array:

0	1	2	3
2	4	5	10

3. Demonstrate the Linear Search algorithm.

Answer:

Algorithm:

Linear_Search(A, element, N)

Step 1: Set Loc := -1
Step 2: Repeat Step 3 for i = 0 to N - 1
Step 3: If (Element = A[i]) then
Set Loc := i
Goto Step 4
End if
End For loop
Step 4: If (Loc >= 0) then
Write('Element found in location', Loc + 1)
Else
Write('Element not found')
Step 5: Exit

4. **Apply the Linear Search algorithm to find the element 10 in the array 20, 40, 10, 30, 60.**

Answer:

Array:

0	1	2	3	4
20	40	10	30	60

Key to Search = 10

- At index 0, the element is 20. Compare with 10 → Not equal.
- At index 1, the element is 40. Compare with 10 → Not equal.
- At index 2, the element is 10. Compare with 10 → They are equal.

Final result: Element found at index 2.

5. How would you solve 45,12,78,34, 89 and 23 using linear search with key= 89?

Answer:

Array:

0	1	2	3	4	5
45	12	78	34	89	23

Key to search = 89

- At index 0, the element is 45. Compare with 89 → Not equal.
- At index 1, the element is 12. Compare with 89 → Not equal.
- At index 2, the element is 78. Compare with 89 → Not equal.
- At index 3, the element is 34. Compare with 89 → Not equal.
- At index 4, the element is 89. Compare with 89 → They are equal

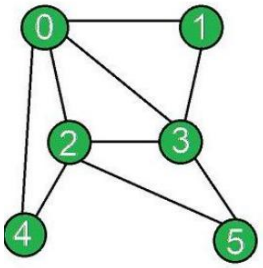
Final result: Element found at index 4

6. Demonstrate the Breadth First Search (BFS) algorithm.

Algorithm:

Step 1: Insert the starting vertex into the Queue and mark it as visited.
Step 2: Repeat until the Queue becomes empty.
Step 3: Remove the front vertex from the Queue and visit it.
Step 4: Find all unvisited neighboring vertices of the removed vertex**.**
Step 5: Mark each neighboring vertex as visited and insert it into the Queue.
Step 6: Repeat the process until all vertices are visited.

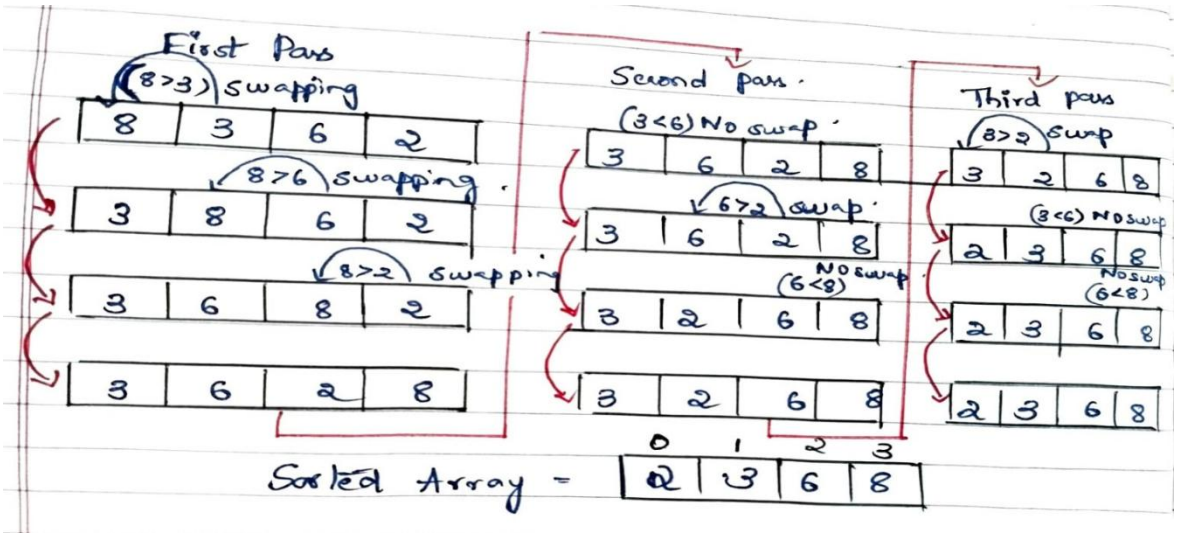
7.Given the following graph, construct its adjacency matrix.



Answer:
Adjacency Matrix:

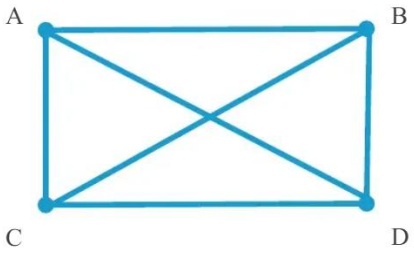
	0	1	2	3	4	5
0	0	1	1	1	1	0
1	1	0	0	1	0	0
2	1	0	0	1	1	1
3	1	1	1	0	0	1
4	1	0	1	0	0	0
5	0	0	1	1	0	0

8.Apply bubble sort to arrange 8, 3, 6, 2 in ascending order. Show the steps.
Answer:



9. Demonstrate the Depth First Search (DFS) algorithm.
Algorithm:
Step 1: Push the starting vertex onto the Stack and mark it as visited.
Step 2: Repeat until the Stack becomes empty.
Step 3: Pop the top vertex of the Stack.
Step 4: If the top vertex has an unvisited neighboring vertex, mark it as visited and push it onto the Stack.
Step 5: If the top vertex has no unvisited neighboring vertices, pop it from the Stack.
Step 6: Continue the process until all vertices are visited.

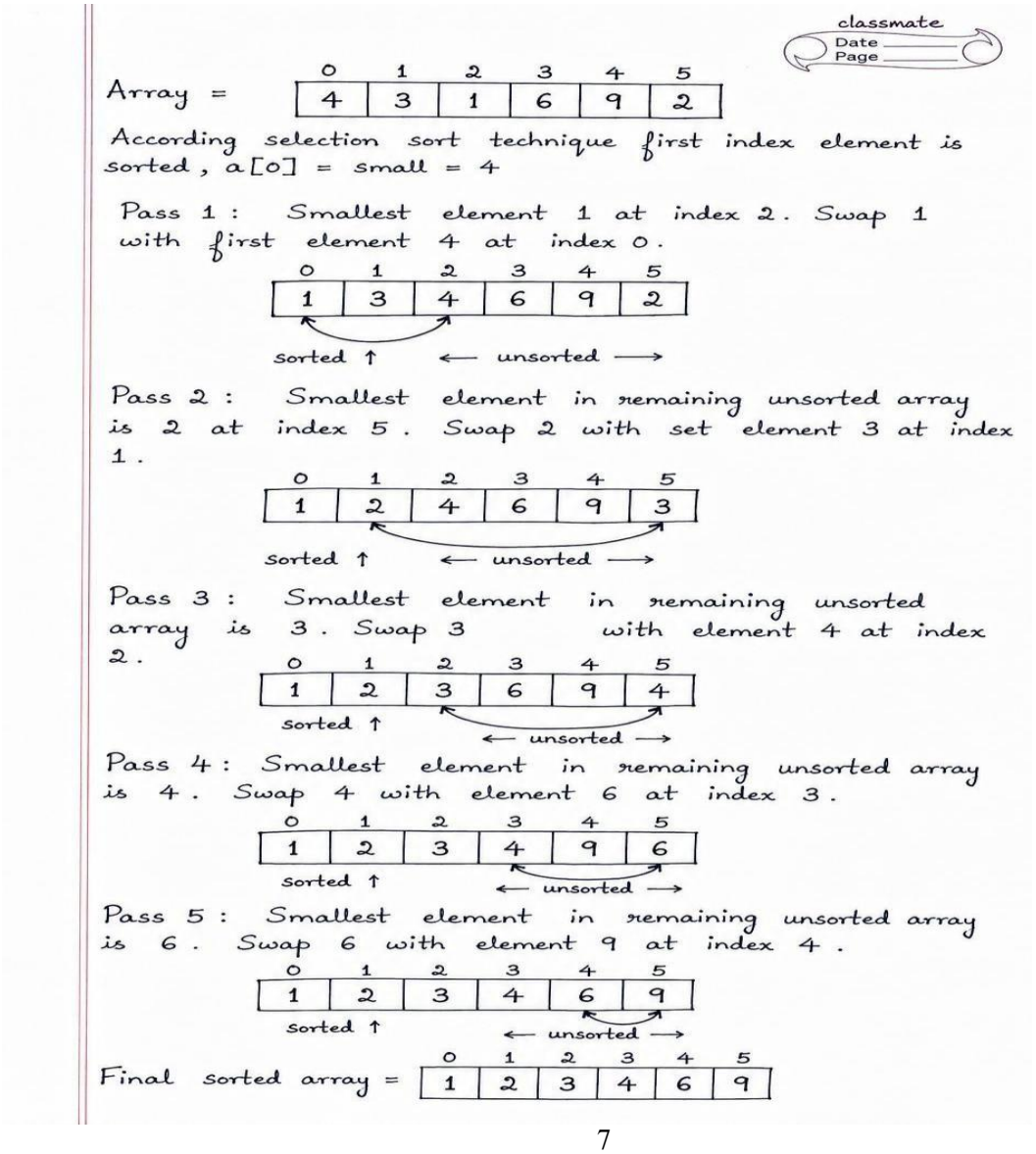
10. Determine the degree of each vertex in the given graph.



Answer:
Vertex A: connected to B, C, and D → **Degree(A) = 3**
Vertex B: connected to A, C, and D → **Degree(B) = 3**
Vertex C: connected to A, B, and D → **Degree(C) = 3**
Vertex D: connected to A, B, and C → **Degree(D) = 3**

IV. Long Answer Questions (5 Marks):

1. How would you solve the problem of sorting the numbers 4, 3, 1, 6, 9 and 2 using selection sort? Explain each step.
Answer:



2. Demonstrate the selection sort algorithm and sort the following values: 10,14,27,33,35,19.

Answer:

Algorithm:

Selection_Sort(A, N)

Step 1: Repeat Steps 2, 3, 4, 6, and 7 for I = 1 to N - 1
Begin

Step 2: Set SMALL := A[I]

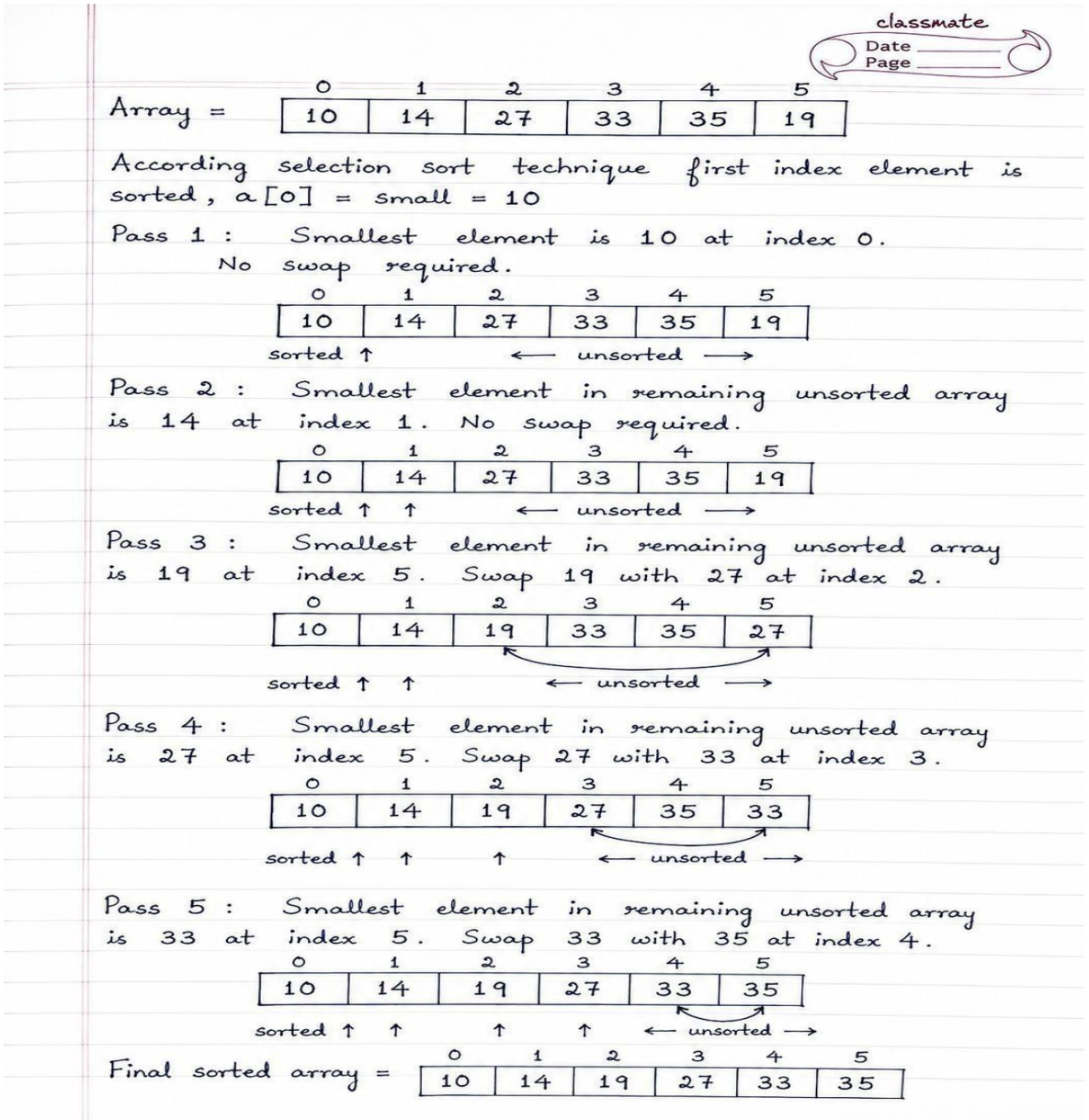
Step 3: Set POSITION := I

Step 4: Repeat Step 5 for J = I + 1 to N
Begin
If A[J] < SMALL then
Set SMALL := A[J]
Set POSITION := J
[End of if structure]
[End of inner for loop]

Step 6: Set A[POSITION] := A[I]

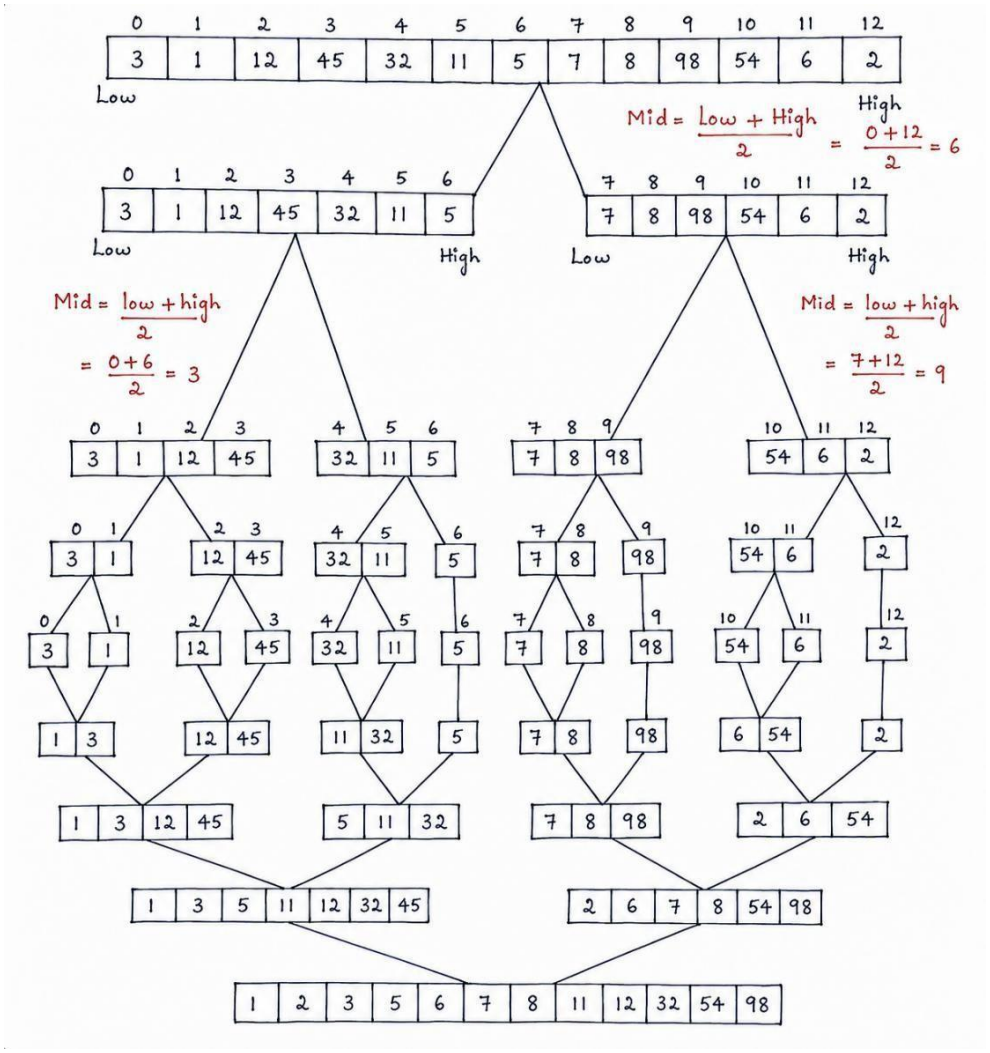
Step 7: Set A[I] := SMALL
[End of outer for loop]

Step 8: Exit



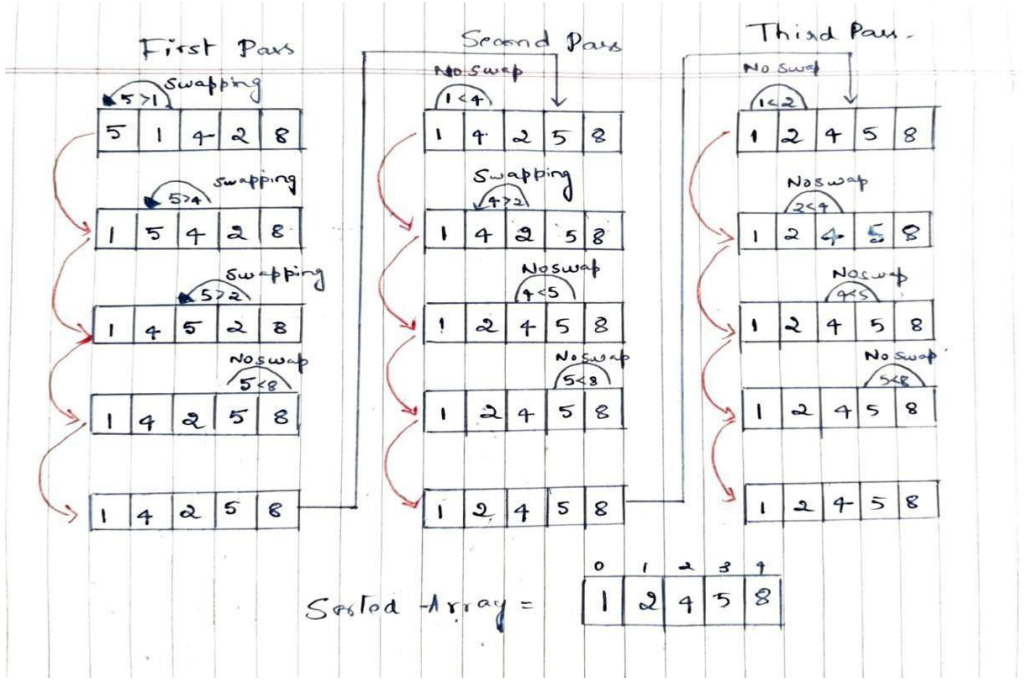
3. How would you use merge sort to arrange the numbers 3, 1, 12, 45, 32, 11, 5, 7, 8, 98, 54, 6, and 2 in ascending order?

Answer:



4. Sort the elements 5,1,4,2 and 8 using bubble sort technique.

Answer:



5. How would you use the binary search to search the key value 45 in the list 6, 12, 17, 23 38, 45, 77, 84 and 90?

Answer:

Array :

	0	1	2	3	4	5	6	7	8
a=	6	12	17	23	38	45	77	84	90

Element to be search = 45 (key)

low = 0

high = 8

$mid = \frac{low + high}{2} = \frac{0+8}{2} = \frac{8}{2} = 4$

Compare key element with middle element

key == a[mid]

45 == a[4]

45 == 38 ⟶ Condition False

key > a[mid]

45 > 38 ⟶ condition True

The element must be present in right half of the array. Eliminate the left half including the middle.

Low = mid + 1

Low = 4 + 1

Low = 5.

	5	6	7	8
a =	45	77	84	90

$mid = \frac{low + high}{2} = \frac{5+8}{2} = \frac{13}{2} = 6.5 \approx 6.$

Compare key element with middle element

key == a[mid]

45 == a[6]

45 == 77 ⟶ condition False.

key > a[mid]

45 > a[6]

45 > 77 ⟶ Condition False

The element must be present in the left half of the array. Eliminate right half of the array including middle.

5

a = [45]

high = mid - 1

high = 6 - 1

high = 5

low = 5

mid = $\frac{low + high}{2} = \frac{5+5}{2} = \frac{10}{2} = 5$

Compare key element with middle element.

key = = a[mid]

45 = = a[5]

45 = = 45

So the element found at index 5. and position = index + 1

5 + 1 = 6;

6. Demonstrate the bubble sort algorithm and sort the following values:8, 2, 9, 1.

Answer:

Algorithm:

Bubble_Sort (A, N)

Step 1: for I= 1 to N-1 do

Step 2: for J = 0 to N-I-1 do

Step 3: If A[J] > A[J+1] then

Temp = A[J]

A[J] = A[j+1]

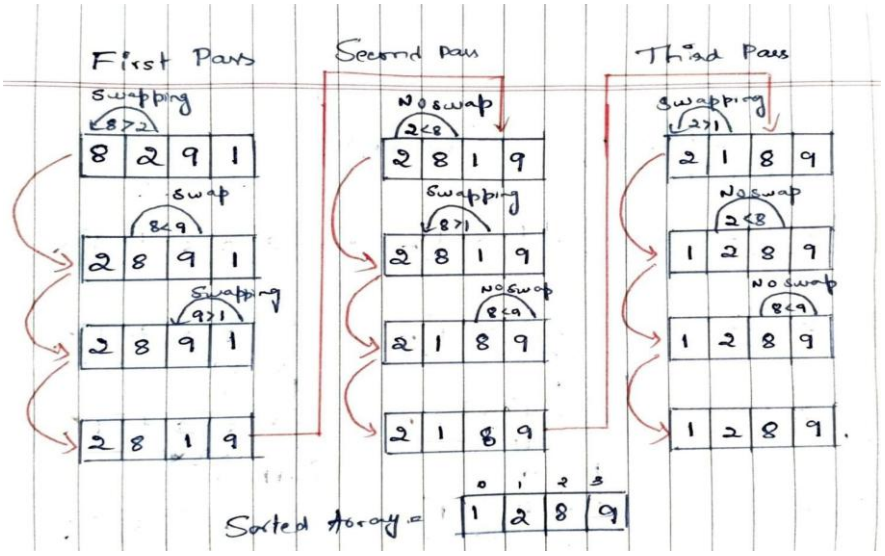
A[J+1] = Temp

[End If]

[End of Step 2 for loop]

[End of Step 1 for loop]

Step 4: Exit



7. Demonstrate the Binary Search algorithm and find the key (16) from the following array: 2, 5, 8, 12,16, 23, 38, 56, 72.

Answer:

Algorithm:

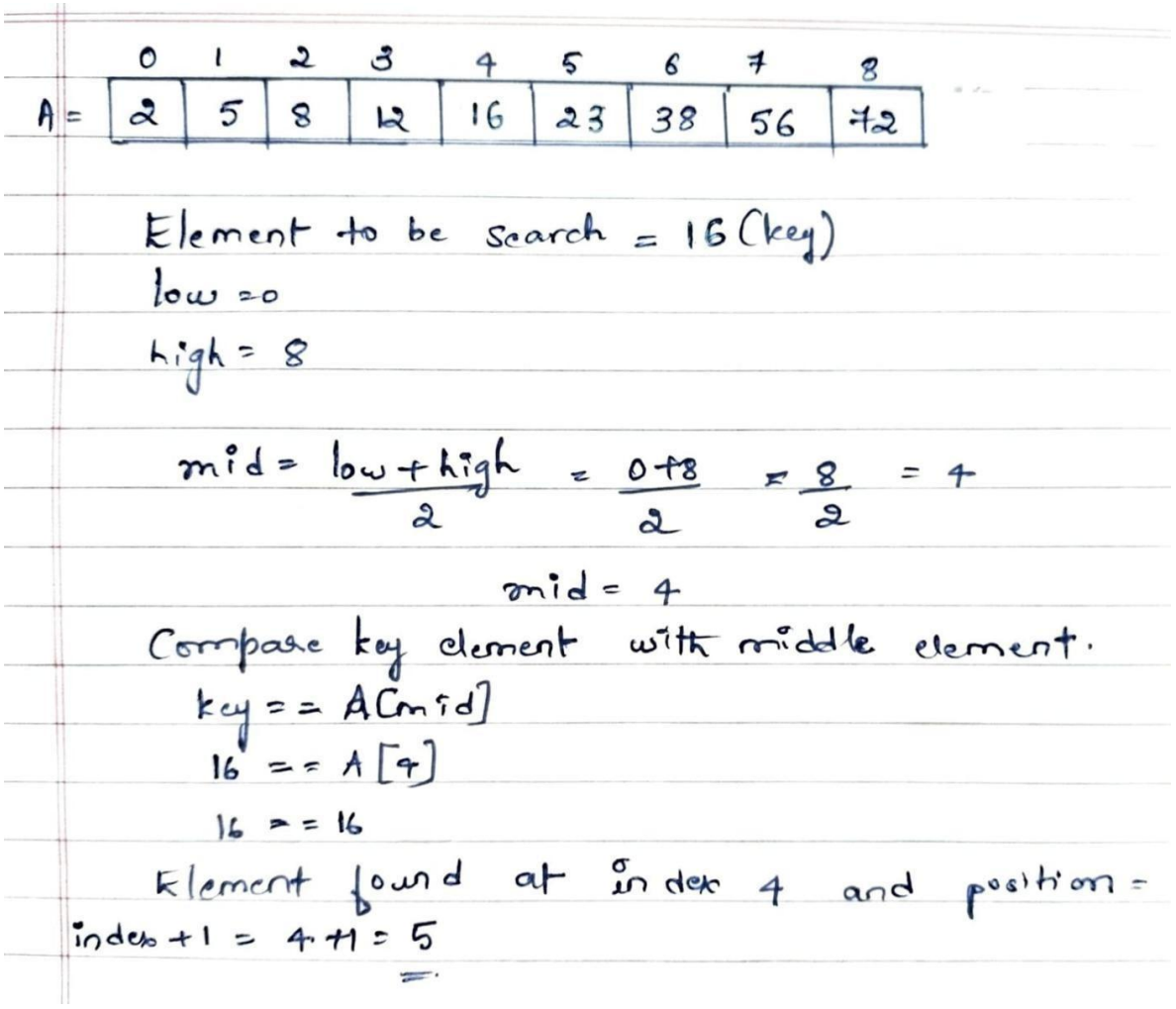
Binary_Search(A, element, N)

Step 1: Set LOW := 0, HIGH := N − 1, LOC := −1

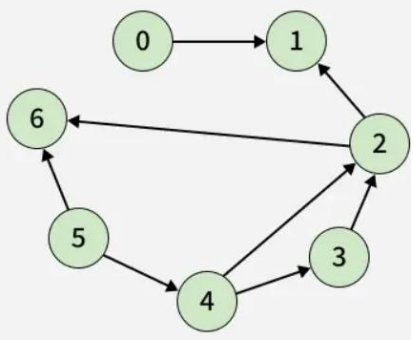
Step 2: Repeat while LOW ≤ HIGH
Set MID := (LOW + HIGH) / 2
If element = A[MID]
Set LOC := MID
Go to Step 3
Else if element < A[MID]
Set HIGH := MID − 1
Else
Set LOW := MID + 1

Step 3: If LOC ≥ 0
Write "Element found at location", LOC
Else
Write "Element not found"

Step 4: EXIT



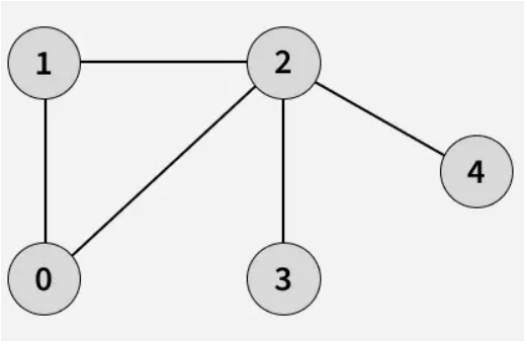
8. Find Indegree and Outdegree for all vertices.



Answer:

Vertex	Indegree	Outdegree
0	0	1
1	2	0
2	2	2
3	1	1
4	1	2
5	0	2
6	2	0

9. Demonstrate the Depth First Search (DFS) algorithm and evaluate the following graph using DFS.



Answer:
Algorithm:
Step 1: Push the starting vertex onto the Stack and mark it as visited.
Step 2: Repeat until the Stack becomes empty.
Step 3: Pop the top vertex of the Stack.
Step 4: If the top vertex has an unvisited neighboring vertex, mark it as visited and push it onto the Stack.
Step 5: If the top vertex has no unvisited neighboring vertices, pop it from the Stack.
Step 6: Continue the process until all vertices are visited.

Step 1: Visit vertex 0 and push it onto the stack.



Visited list = | 0 | | | | |

Step 2: Move to the leftmost unvisited adjacent vertex 1. Visit 1 and push it onto the stack.



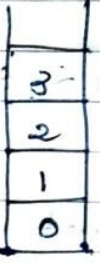
Visited list = | 0 | 1 | | | |

Step 3: Move to the unvisited adjacent vertex 2. Visit 2 and push it onto the stack.



Visited list = | 0 | 1 | 2 | | |

Step 4: Move to the unvisited adjacent vertex 3. Visit 3 and push it onto the stack.



Visited list = | 0 | 1 | 2 | 3 | |

Step 5: Vertex 3 has no adjacent vertex. Pop 3 from the stack.



Visited list =

0	1	2	3	

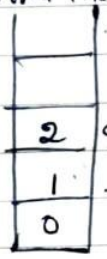
Step 6: Move back to vertex 2. Its next unvisited vertex 4. Visit 4 and push it onto stack.



Visited list =

0	1	2	3	4

Step 7: Vertex 4 has no adjacent vertex. Pop 4 from the stack.



visited list =

0	1	2	3	4

Step 8: Vertex 2 has no adjacent vertices. Pop 2



visited list =

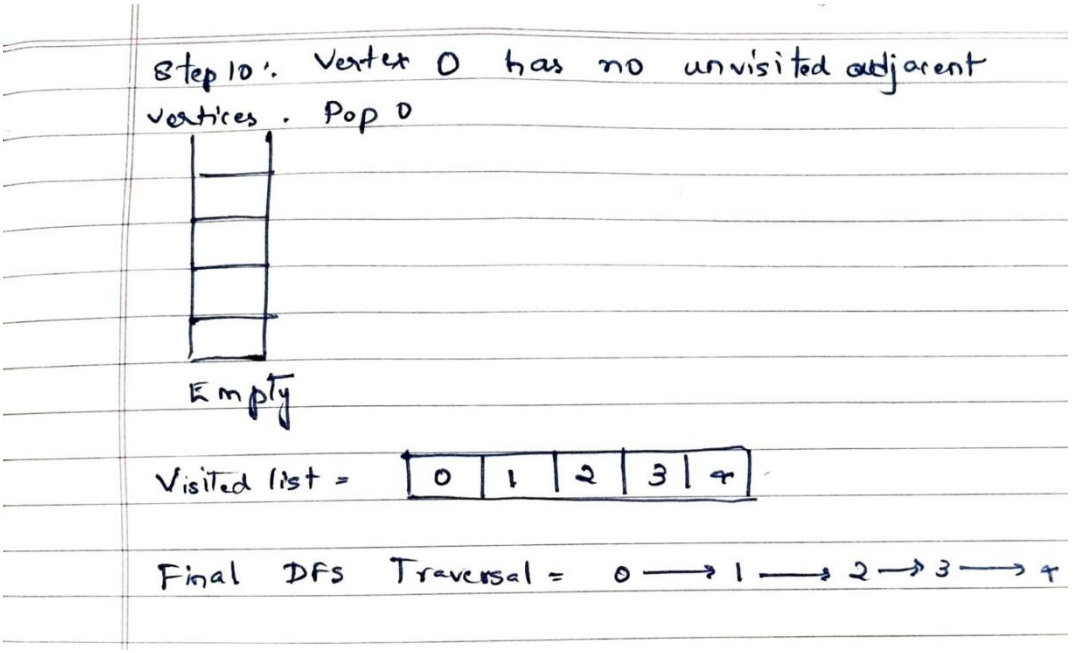
0	1	2	3	4

Step 9: Vertex 1 has no adjacent vertices. Pop 1



Visited list =

0	1	2	3	4



10. Demonstrate the Linear Search algorithm to find the element 90 in the array [6, 3, 8, 1, 90, 33] and explain each step.

Answer:

Algorithm: Linear_Search(A,element,N)

A is an array of N elements and element is the element being searched in the array.

Step 1: Set Loc:= -1
Step 2: Repeat step3 For i=0 to n-1
Step 3: If (Element==A[i]) then
begin
i. Set loc:= i
ii. Goto step 4
[End if]
[End for loop]
Step 4: If (loc>=0) then
i. WRITE("element is fount at location", Loc);
ii. Else
iii. WRITE("Not found")
Step 5: Exit

Solution / Trace

Target Element: 90

Given Array: [6, 3, 8, 1, 90, 33]

Step-by-Step Execution:

1. Index 0: Value = 6 ≠ 90 (Mismatch)
2. Index 1: Value = 3 ≠ 90 (Mismatch)
3. Index 2: Value = 8 ≠ 90 (Mismatch)
4. Index 3: Value = 1 ≠ 90 (Mismatch)
5. Index 4: Value = 90 = 90 (Match Found!)

Final Result:

- **Element Found: Yes (90)**
- **Index Position: 4 (5th element in the array)**